

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ



BÁO CÁO BÀI TẬP LỚN

Đề tài: LoRa đa chặng

Sinh viên thực hiện: Phạm Nam Phúc - 202414944

Hoàng Vĩ Kỳ - 202414928

Trần Minh Tuyên - 202414958

Đỗ Đức Lợi - 202414932

Mã lớp: 168260

Môn học: Lý thuyết thông tin

Giảng viên hướng dẫn: Nguyễn Hữu Phát

MỤC LỤC

Giới Thiệu Đề Tài	6
1.1 Bối cảnh và nhu cầu truyền thông IoT tầm xa	6
1.2 Hạn chế của mô hình truyền LoRa một chặng	7
1.3 Động lực xây dựng hệ thống LoRa đa chặng.....	7
1.4 Mục tiêu của đề tài	8
1.5 Đối tượng và phạm vi thực hiện.....	9
1.6 Phương pháp tiếp cận.....	9
1.7 Các yêu cầu đặt ra cho hệ thống.....	10
1.8 Các chỉ số đánh giá dự kiến	11
1.9 Ý nghĩa khoa học và thực tiễn của đề tài	11
1.10 Đóng góp chính của đề tài	12
1.11 Cấu trúc báo cáo	12
2 CƠ SỞ LÝ THUYẾT.....	13
2.1 Tổng quan về hệ thống truyền tin.....	13
2.2 Công nghệ LoRa	14
2.3 LoRa và LoRaWAN.....	15
2.4 Mạng truyền dữ liệu đa chặng	16
2.5 Entropy trong lý thuyết thông tin	16
2.6 Mã hóa nguồn và thuật toán Huffman	17
2.7 Tỷ lệ nén.....	18
2.8 Kiểm tra lỗi CRC	18
2.9 ACK và cơ chế truyền lại ARQ.....	19
2.10 RSSI và SNR.....	19
2.11 Packet loss, delay và throughput	20
2.12 Bluetooth trong vai trò giao diện đầu vào	20
2.13 Kết luận chương.....	21
3 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	21

3.1	Sơ đồ khối tổng quát của hệ thống	21
3.2	Vai trò của từng Node trong mạng đa chặng (Mesh Network).....	22
3.3	Thiết kế phần cứng và sơ đồ kết nối	22
3.4	Luồng hoạt động tổng quát (System Workflow)	23
4	THIẾT KẾ GIAO THỨC VÀ THUẬT TOÁN	24
4.1	Cấu trúc Giao thức và Gói tin (LoRa Packet Protocol)	24
4.2	Thuật toán Phân mảnh và Ghép mảnh (Fragmentation & Reassembly)	25
4.3	Quản lý luồng bằng Hàng đợi thu (RX Queue) và Hàm Delay	26
4.4	Thuật toán LBT (Listen Before Talk) và CSMA/CA Random Backoff.....	27
4.5	Thuật toán chống lặp (Deduplication) trong Mạng Flooding	28
4.6	Áp dụng Mã hóa Nguồn: Nén Static Huffman cấp độ Bit.....	28
4.7	Giao tiếp ngoại vi và Kiến trúc Bluetooth Asynchronous	29
4.8	Luồng Xử lý Sửa lỗi (FEC) và Phát hiện lỗi phần cứng (Hardware CRC)	29
5	CÀI ĐẶT HỆ THỐNG VÀ KIẾN TRÚC MÃ NGUỒN	30
5.1	Môi trường Phát triển (Development Environment)	30
5.2	Tổ chức Mã nguồn (Source Code Structure)	30
5.3	Quy trình Cấu hình cho từng Node (Node Provisioning).....	31
6	KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ.....	32
6.1	Đánh giá Thuật toán Mã hóa nguồn (Entropy & Compression Ratio)	32
6.2	Đánh giá Chất lượng kênh truyền vô tuyến (RSSI & SNR).....	33
6.3	Đánh giá Thuật toán CSMA/CA và Tỷ lệ mất gói (Packet Loss).....	36
6.4	Đánh giá Độ trễ khứ hồi (Delay / RTT)	38
7	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	40
7.1	Đánh giá những kết quả đạt được	40
7.2	Các hạn chế còn tồn tại	40
7.3	Định hướng phát triển trong tương lai.....	41

DANH SÁCH HÌNH ẢNH

Hình 0.1. Mô hình tổng quát hệ thống truyền một chặng.....	7
Hình 0.2. Mô hình tổng quát hệ thống truyền đa chặng.....	8
Hình 2.1. Mô hình hệ thống truyền tin tổng quát.....	13
Hình 2.2. Mô hình truyền một chặng	16
Hình 2.3. Mô hình truyền đa chặng.....	16
Hình 2.4. Mô hình mở rộng hệ thống truyền LoRa đa chặng.....	20
Hình 3.1. Sơ đồ khối tổng quát của hệ thống	21
Hình 6.1. Kết quả đo xuyên 1 bức tường	33
Hình 6.2. Kết quả đo giữa tầng 2 và tầng 5	34
Hình 6.3. Kết quả đo với khoảng cách 10m (trái) và 20m (phải).....	35
Hình 6.4. Kết quả đo với khoảng cách 50m (trái) và 100m (phải).....	35
Hình 6.5. Test trường hợp tắt thuật toán CSMA/CA	37
Hình 6.6. Test trường hợp bật CSMA/CA.....	38
Hình 6.7. Kết quả đo RTT ở khoảng cách 50m(trái) và 100m(phải)	39
Hình 6.8. Kết quả đo RTT ở khoảng cách 200m có relay node	39

DANH SÁCH BẢNG

Bảng 1.1. Các yêu cầu chính của hệ thống	10
Bảng 1.2. Các chỉ số đánh giá trong đề tài	11
Bảng 2.1. Tham số quan trọng của LoRa	14
Bảng 3.1. Sơ đồ kết nối Chân (Pinout) giữa ESP32 và LoRa SX1278	23
Bảng 4.1. Bảng danh sách các trường trong gói tin	24
Bảng 6.1. Kết quả đo đặc Entropy và Tỷ lệ nén Huffman	32

Giới Thiệu Đề Tài

1.1 Bối cảnh và nhu cầu truyền thông IoT tầm xa

Trong những năm gần đây, Internet of Things (IoT) ngày càng được ứng dụng rộng rãi trong nhiều lĩnh vực như nông nghiệp thông minh, giám sát môi trường, nhà thông minh, thành phố thông minh, logistics, cảnh báo thiên tai và các hệ thống cảm biến phân tán. Điểm chung của các hệ thống này là cần thu thập dữ liệu từ nhiều thiết bị đặt tại các vị trí khác nhau, sau đó truyền dữ liệu về một điểm xử lý trung tâm hoặc chia sẻ giữa các node trong mạng.

Đối với các ứng dụng trong phạm vi nhỏ, những công nghệ truyền thông không dây như Wi-Fi, Bluetooth hoặc Zigbee có thể đáp ứng tốt nhu cầu kết nối. Tuy nhiên, khi phạm vi triển khai mở rộng ra các khu vực lớn hơn như cánh đồng, khu công nghiệp, nhà xưởng, vùng đồi núi hoặc khu vực thiếu hạ tầng mạng, các công nghệ này bắt đầu bộc lộ hạn chế. Wi-Fi có tốc độ truyền cao nhưng phạm vi phủ sóng không lớn và tiêu thụ năng lượng tương đối cao. Bluetooth phù hợp với kết nối khoảng cách gần. Zigbee hỗ trợ mạng mesh nhưng khoảng cách mỗi liên kết thường bị giới hạn. Trong khi đó, mạng di động có vùng phủ rộng nhưng phụ thuộc vào hạ tầng nhà mạng và có thể làm tăng chi phí vận hành.

Từ nhu cầu đó, các công nghệ LPWAN (Low Power Wide Area Network) được phát triển nhằm phục vụ các hệ thống cần truyền dữ liệu tốc độ thấp, khoảng cách xa và tiêu thụ năng lượng thấp. LoRa là một trong những công nghệ tiêu biểu thuộc nhóm LPWAN. Công nghệ này phù hợp với các ứng dụng truyền dữ liệu nhỏ, không yêu cầu băng thông lớn nhưng cần khả năng hoạt động ổn định trong phạm vi rộng.

Trong các hệ thống cảm biến thực tế, dữ liệu truyền đi thường là các bản tin có kích thước nhỏ như nhiệt độ, độ ẩm, trạng thái thiết bị, tín hiệu cảnh báo hoặc thông điệp điều khiển. Đây là kiểu dữ liệu phù hợp với LoRa. Nhờ ưu điểm về khoảng cách truyền, khả năng chống nhiễu và chi phí phần cứng thấp, LoRa trở thành một lựa chọn phù hợp cho các bài toán IoT tầm xa.

Tuy nhiên, khả năng truyền xa của LoRa không phải là tuyệt đối. Trong môi trường thực tế, tín hiệu vô tuyến luôn chịu ảnh hưởng bởi khoảng cách, vật cản, địa hình, nhiễu điện từ, chất lượng anten và cấu hình truyền. Vì vậy, để xây dựng một hệ thống truyền dữ liệu ổn định hơn trong phạm vi rộng, cần xem xét các mô hình truyền thông linh hoạt hơn so với truyền trực tiếp một chặng.

1.2 Hạn chế của mô hình truyền LoRa một chặng

Trong mô hình truyền LoRa một chặng, node nguồn truyền dữ liệu trực tiếp đến node đích. Mô hình này đơn giản, dễ triển khai và phù hợp khi hai node nằm trong vùng phủ sóng của nhau. Tuy nhiên, khi khoảng cách giữa hai node tăng lên hoặc môi trường có nhiều vật cản, chất lượng liên kết có thể suy giảm đáng kể. Khi đó, gói tin có thể bị lỗi, bị mất hoặc không được node đích nhận đúng.

Một hệ thống truyền một chặng có thể được mô tả như sau:



Hình 0.1. Mô hình tổng quát hệ thống truyền một chặng

Với mô hình này, toàn bộ quá trình truyền phụ thuộc vào chất lượng của liên kết trực tiếp giữa node nguồn và node đích. Nếu liên kết yếu, hệ thống không có node trung gian hỗ trợ chuyển tiếp. Điều này làm giới hạn vùng phủ sóng và giảm khả năng mở rộng của mạng.

Trong thực tế, có nhiều tình huống mà truyền trực tiếp không phải là lựa chọn tối ưu. Ví dụ, một node cảm biến có thể được đặt ở vị trí xa gateway; giữa hai node có thể có tường, cây cối, thiết bị kim loại hoặc địa hình che khuất; hoặc hệ thống cần bao phủ một khu vực rộng hơn khả năng truyền trực tiếp của từng node. Trong các trường hợp đó, việc tăng công suất phát không phải lúc nào cũng phù hợp vì có thể làm tăng tiêu thụ năng lượng, tăng nhiễu và bị giới hạn bởi quy định tần số.

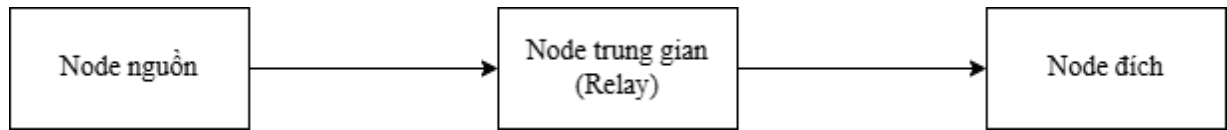
Bên cạnh vấn đề khoảng cách, LoRa còn có đặc điểm tốc độ truyền không cao. Khi truyền gói tin dài, thời gian chiếm kênh truyền tăng lên. Nếu nhiều node cùng truyền trong cùng một khoảng thời gian, nguy cơ va chạm tín hiệu cũng tăng. Vì vậy, hệ thống truyền LoRa cần được thiết kế sao cho gói tin có kích thước hợp lý, có cơ chế kiểm soát lỗi và có khả năng đánh giá chất lượng truyền.

Những hạn chế trên là cơ sở để nghiên cứu mô hình truyền LoRa đa chặng, trong đó dữ liệu không nhất thiết phải đi trực tiếp từ node nguồn đến node đích mà có thể được chuyển tiếp qua các node trung gian.

1.3 Động lực xây dựng hệ thống LoRa đa chặng

Mô hình truyền đa chặng cho phép dữ liệu được truyền qua một hoặc nhiều node trung gian trước khi đến được node đích hoặc đến toàn bộ mạng. Thay vì yêu cầu node nguồn phải truyền trực tiếp đến mọi node khác, hệ thống có thể tận dụng các node nằm giữa để mở rộng phạm vi truyền.

Mô hình tổng quát có thể mô tả như sau:



Hình 0.2. Mô hình tổng quát hệ thống truyền đa chặng

Cách tiếp cận này có nhiều ưu điểm. Thứ nhất, vùng phủ sóng của toàn hệ thống có thể được mở rộng mà không cần mỗi node phải truyền với công suất lớn. Thứ hai, các liên kết ngắn hơn giữa các node có thể ổn định hơn so với một liên kết trực tiếp quá xa. Thứ ba, mô hình đa chặng phù hợp với các hệ thống cảm biến phân tán, nơi các node được bố trí rải rác trên một khu vực rộng.

Tuy nhiên, truyền đa chặng cũng đặt ra các vấn đề kỹ thuật mới. Khi một node nhận được gói tin và phát lại, các node khác có thể nhận cùng một gói nhiều lần. Nếu không có cơ chế kiểm soát, gói tin có thể bị relay lặp lại liên tục, gây lãng phí tài nguyên và làm tăng khả năng tắc nghẽn kênh truyền. Ngoài ra, vì LoRa là môi trường truyền vô tuyến chia sẻ, việc nhiều node cùng phát lại có thể dẫn đến va chạm tín hiệu.

Do đó, một hệ thống LoRa đa chặng cần có cơ chế nhận diện gói tin, tránh xử lý trùng lặp, kiểm tra lỗi, quản lý dữ liệu dài và đánh giá chất lượng truyền. Đây chính là các vấn đề trọng tâm mà đề tài hướng đến.

Trong đề tài này, hệ thống được xây dựng theo hướng mạng nhiều node có khả năng broadcast và relay gói tin. Mỗi node có thể đóng vai trò gửi, nhận hoặc chuyển tiếp dữ liệu. Hệ thống không chỉ dừng ở việc truyền nhận cơ bản mà còn được bổ sung các thành phần phục vụ đánh giá truyền tin như đo cường độ tín hiệu, tỷ số tín hiệu trên nhiễu, tỷ lệ nén và tỷ lệ mất gói ước lượng. Các nội dung kỹ thuật chi tiết sẽ được trình bày trong các chương sau.

1.4 Mục tiêu của đề tài

Mục tiêu tổng quát của đề tài là thiết kế và triển khai một hệ thống truyền dữ liệu LoRa đa chặng trên phần cứng thật, sử dụng vi điều khiển ESP32 kết hợp với module LoRa SX1278. Hệ thống cần có khả năng gửi, nhận và chuyển tiếp dữ liệu giữa nhiều node, đồng thời cung cấp một số chỉ số phục vụ đánh giá chất lượng truyền tin.

Các mục tiêu cụ thể của đề tài bao gồm:

- Xây dựng mô hình mạng LoRa gồm nhiều node độc lập.
- Thiết kế mỗi node dựa trên ESP32 và module LoRa SX1278.
- Cấu hình truyền thông LoRa phù hợp cho mô hình thử nghiệm đa chặng.
- Xây dựng cơ chế truyền dữ liệu dạng broadcast trong mạng.
- Cho phép node trung gian nhận và chuyển tiếp gói tin.
- Xây dựng cơ chế hạn chế xử lý trùng lặp gói tin.

- Dữ liệu truyền là chuỗi văn bản do người dùng nhập. Trong giai đoạn thử nghiệm, dữ liệu được nhập qua Serial Monitor; trong định hướng hoàn thiện, node gửi có thể nhận dữ liệu từ điện thoại thông qua Bluetooth.
- Hỗ trợ xử lý thông điệp có kích thước lớn hơn payload của một gói đơn.
- Tích hợp cơ chế kiểm tra lỗi để phát hiện packet không hợp lệ.
- Áp dụng kỹ thuật nén dữ liệu khi việc nén mang lại lợi ích.
- Ghi nhận một số chỉ số đánh giá như RSSI, SNR, entropy, tỷ lệ nén và tỷ lệ mất gói ước lượng.
- Thử nghiệm truyền dữ liệu thật giữa các node và phân tích kết quả thu được.

Các mục tiêu trên được đặt ra nhằm đảm bảo đề tài không chỉ có phần demo truyền dữ liệu thực tế mà còn có cơ sở phân tích theo hướng của môn Lý thuyết thông tin.

1.5 Đối tượng và phạm vi thực hiện

Đối tượng thực hiện của đề tài là hệ thống truyền dữ liệu không dây sử dụng công nghệ LoRa, trong đó các node có khả năng hoạt động độc lập và tham gia vào quá trình truyền dữ liệu đa chặng. Mỗi node trong hệ thống gồm một vi điều khiển ESP32 và một module LoRa SX1278. Các node sử dụng cùng một chương trình chính, chỉ khác nhau ở mã định danh node được cấu hình khi biên dịch.

Trong phạm vi bài tập lớn, hệ thống được triển khai với mô hình gồm ba node. Dữ liệu truyền là chuỗi văn bản được nhập từ điện thoại thông qua Bluetooth. Hệ thống tập trung vào mô hình truyền broadcast, trong đó một node phát thông điệp và các node còn lại có thể nhận hoặc chuyển tiếp thông điệp.

Đề tài tập trung vào việc xây dựng giao thức truyền dữ liệu ở mức ứng dụng trên nền LoRa point-to-point. Hệ thống không triển khai đầy đủ chuẩn LoRaWAN, không sử dụng gateway kết nối Internet và chưa xây dựng thuật toán định tuyến tối ưu theo bảng định tuyến. Ngoài ra, đề tài chưa tích hợp cơ chế bảo mật hoặc mã hóa nội dung payload.

Phạm vi này được lựa chọn để phù hợp với thời gian và yêu cầu của bài tập lớn, đồng thời vẫn đảm bảo hệ thống có đầy đủ các thành phần quan trọng của một mô hình truyền tin thực tế như phần cứng, truyền nhận dữ liệu, xử lý lỗi, chuyển tiếp gói tin và đánh giá một số chỉ số truyền thông.

1.6 Phương pháp tiếp cận

Đề tài được thực hiện theo hướng kết hợp giữa nghiên cứu lý thuyết, thiết kế hệ thống và triển khai thực nghiệm trên phần cứng. Trước hết, nhóm tìm hiểu các đặc điểm cơ bản của công nghệ LoRa, mô hình truyền đa chặng và các chỉ số thường dùng để đánh giá chất lượng truyền tin. Sau đó, hệ thống phần cứng được xây dựng dựa trên ESP32 và module LoRa SX1278.

Ở phần phần mềm, chương trình được phát triển trên nền PlatformIO với framework Arduino. Mã nguồn được tổ chức thành các thành phần chính như khởi tạo LoRa, xử lý packet và xử lý

nén dữ liệu. Việc chia tách mã nguồn theo chức năng giúp chương trình dễ theo dõi, dễ mở rộng và thuận tiện cho quá trình giải thích trong báo cáo.

Sau khi hoàn thiện chương trình, các node được nạp firmware với mã định danh khác nhau. Quá trình thử nghiệm được thực hiện bằng cách nhập dữ liệu từ Serial Monitor (trong định hướng hoàn thiện, node gửi có thể nhận dữ liệu từ điện thoại thông qua Bluetooth) ở một node, sau đó quan sát log nhận, log chuyển tiếp và các chỉ số đo được ở các node còn lại. Từ kết quả thực nghiệm, nhóm tiến hành đánh giá khả năng truyền dữ liệu, hiệu quả nén và chất lượng liên kết vô tuyến.

Quy trình thực hiện tổng quát của đề tài gồm các bước sau:

- Tìm hiểu công nghệ LoRa và mô hình truyền dữ liệu đa chặng.
- Lựa chọn phần cứng ESP32 và module LoRa SX1278.
- Thiết kế sơ đồ kết nối phần cứng.
- Xây dựng môi trường phát triển bằng PlatformIO.
- Cài đặt chương trình truyền nhận dữ liệu cho các node.
- Tích hợp các cơ chế kiểm tra lỗi, chuyển tiếp và xử lý dữ liệu.
- Nạp chương trình lên các node thực tế.
- Thực hiện truyền dữ liệu thử nghiệm.
- Ghi nhận log và các chỉ số đánh giá.
- Phân tích kết quả, nhận xét hạn chế và đề xuất hướng phát triển.

1.7 Các yêu cầu đặt ra cho hệ thống

Để đáp ứng mục tiêu của đề tài, hệ thống cần thỏa mãn một số yêu cầu cơ bản. Các yêu cầu này được xây dựng dựa trên đặc điểm của mạng LoRa đa chặng và tiêu chí đánh giá của bài tập lớn. Bảng dưới đây trình bày các yêu cầu chính của hệ thống:

Bảng 0.1. Các yêu cầu chính của hệ thống

Yêu cầu	Mô tả
Truyền dữ liệu thật	Hệ thống hoạt động trên phần cứng ESP32 và module LoRa SX1278
Nhiều node	Hệ thống gồm nhiều node có mã định danh khác nhau
Truyền đa chặng	Node trung gian có khả năng nhận và chuyển tiếp gói tin
Broadcast	Một node có thể gửi thông điệp đến toàn mạng
Chống trùng lặp	Gói tin đã xử lý không được xử lý lặp lại nhiều lần
Kiểm tra lỗi	Packet cần có cơ chế phát hiện lỗi trong quá trình truyền
Hỗ trợ dữ liệu dài	Thông điệp dài cần được xử lý phù hợp với giới hạn payload
Tối ưu dữ liệu	Hệ thống có khả năng giảm kích thước dữ liệu khi phù hợp
Ghi nhận chỉ số	Hệ thống cần ghi nhận một số thông số phục vụ đánh giá thực nghiệm
Dễ mở rộng	Kiến trúc phần mềm cần đủ rõ ràng để có thể phát triển thêm

Bên cạnh các yêu cầu chức năng, hệ thống cũng cần đáp ứng một số yêu cầu phi chức năng như hoạt động ổn định trong quá trình demo, dễ quan sát log trên Serial Monitor (định hướng sau này là trên điện thoại thông qua Bluetooth), cấu hình node rõ ràng và có khả năng giải thích được nguyên lý hoạt động.

1.8 Các chỉ số đánh giá dự kiến

Để đánh giá hệ thống truyền tin, đề tài sử dụng một số chỉ số phản ánh chất lượng dữ liệu, chất lượng kênh truyền và hiệu quả xử lý. Các chỉ số này được ghi nhận từ chương trình hoặc tính toán dựa trên log thực nghiệm.

Bảng 0.2. Các chỉ số đánh giá trong đề tài

Chỉ số	Ý nghĩa
RSSI	Cường độ tín hiệu nhận được tại node thu
SNR	Tỷ số tín hiệu trên nhiễu, phản ánh chất lượng kênh truyền
Entropy	Mức độ thông tin trung bình của dữ liệu nguồn
Compression ratio	Tỷ lệ giữa kích thước sau nén và kích thước ban đầu
Packet loss	Tỷ lệ gói hoặc fragment truyền thất bại
Số fragment	Số mảnh cần truyền cho một thông điệp
Delay/RTT	Độ trễ truyền hoặc thời gian phản hồi trong trường hợp có ACK

Các chỉ số trên sẽ được giải thích chi tiết hơn ở Chương 2 và được sử dụng trong phần thực nghiệm ở Chương 6. Trong Chương 1, các chỉ số này chỉ được nêu nhằm xác định định hướng đánh giá của đề tài.

1.9 Ý nghĩa khoa học và thực tiễn của đề tài

Về mặt khoa học, đề tài giúp liên hệ giữa các kiến thức lý thuyết và một hệ thống truyền tin thực tế. Các nội dung như mã hóa nguồn, phát hiện lỗi, chất lượng kênh truyền và mất gói không chỉ được trình bày dưới dạng khái niệm mà còn được áp dụng trong quá trình thiết kế và thử nghiệm hệ thống. Điều này giúp làm rõ vai trò của Lý thuyết thông tin trong các hệ thống truyền thông không dây.

Về mặt thực tiễn, đề tài cho thấy khả năng xây dựng một mạng truyền dữ liệu không dây tầm xa với chi phí thấp. Mô hình LoRa đa chặng có thể được phát triển cho các ứng dụng như giám sát nông nghiệp, thu thập dữ liệu môi trường, cảnh báo từ xa hoặc kết nối các thiết bị cảm biến trong khu vực rộng. Với cách tiếp cận nhiều node, hệ thống có tiềm năng mở rộng phạm vi truyền mà không cần phụ thuộc hoàn toàn vào hạ tầng mạng sẵn có.

Ngoài ra, quá trình thực hiện đề tài cũng giúp người thực hiện rèn luyện nhiều kỹ năng quan trọng như thiết kế hệ thống nhúng, cấu hình truyền thông SPI, lập trình vi điều khiển, làm việc với module vô tuyến, thiết kế giao thức truyền dữ liệu, đọc log thực nghiệm và phân tích hiệu năng hệ thống.

1.10 Đóng góp chính của đề tài

Các đóng góp chính của đề tài có thể tóm tắt như sau:

- Xây dựng được hệ thống truyền dữ liệu thật sử dụng ESP32 và module LoRa SX1278.
- Tổ chức được mô hình nhiều node có khả năng gửi, nhận và chuyển tiếp dữ liệu.
- Thiết kế được cơ chế truyền broadcast phù hợp với mô hình thử nghiệm.
- Tích hợp cơ chế hạn chế xử lý trùng lặp trong quá trình relay.
- Hỗ trợ xử lý thông điệp văn bản có kích thước vượt quá giới hạn payload của một gói đơn.
- Tích hợp kiểm tra lỗi nhằm loại bỏ packet không hợp lệ.
- Áp dụng kỹ thuật nén dữ liệu để cải thiện hiệu quả truyền trong trường hợp phù hợp.
- Ghi nhận được các chỉ số phục vụ đánh giá như RSSI, SNR, entropy, tỷ lệ nén và tỷ lệ mất gói ước lượng.
- Xây dựng nền tảng có thể mở rộng cho các hướng phát triển như định tuyến tối ưu, đo throughput tự động, bảo mật payload hoặc giao diện giám sát.

1.11 Cấu trúc báo cáo

Phần còn lại của báo cáo được tổ chức như sau:

Chương 2 trình bày cơ sở lý thuyết liên quan đến đề tài, bao gồm công nghệ LoRa, mô hình truyền đa chặng, các khái niệm về entropy, mã hóa nguồn, kiểm tra lỗi và các chỉ số đánh giá chất lượng truyền tin.

Chương 3 trình bày phân tích và thiết kế hệ thống, bao gồm kiến trúc tổng thể, sơ đồ khối, thiết kế phần cứng, vai trò của từng node và luồng hoạt động tổng quát.

Chương 4 trình bày thiết kế giao thức và thuật toán, bao gồm định dạng gói tin, cơ chế gửi nhận, chuyển tiếp, chống trùng lặp, phân mảnh, ghép mảnh và xử lý dữ liệu.

Chương 5 trình bày quá trình cài đặt hệ thống, môi trường phát triển, cấu trúc mã nguồn, cấu hình cho từng node và các thành phần chính trong chương trình.

Chương 6 trình bày kết quả thực nghiệm và đánh giá, bao gồm kịch bản thử nghiệm, log truyền nhận, các chỉ số đo được và phân tích hiệu quả của hệ thống.

Chương 7 trình bày kết luận, các hạn chế còn tồn tại và hướng phát triển trong tương lai.

2 CƠ SỞ LÝ THUYẾT

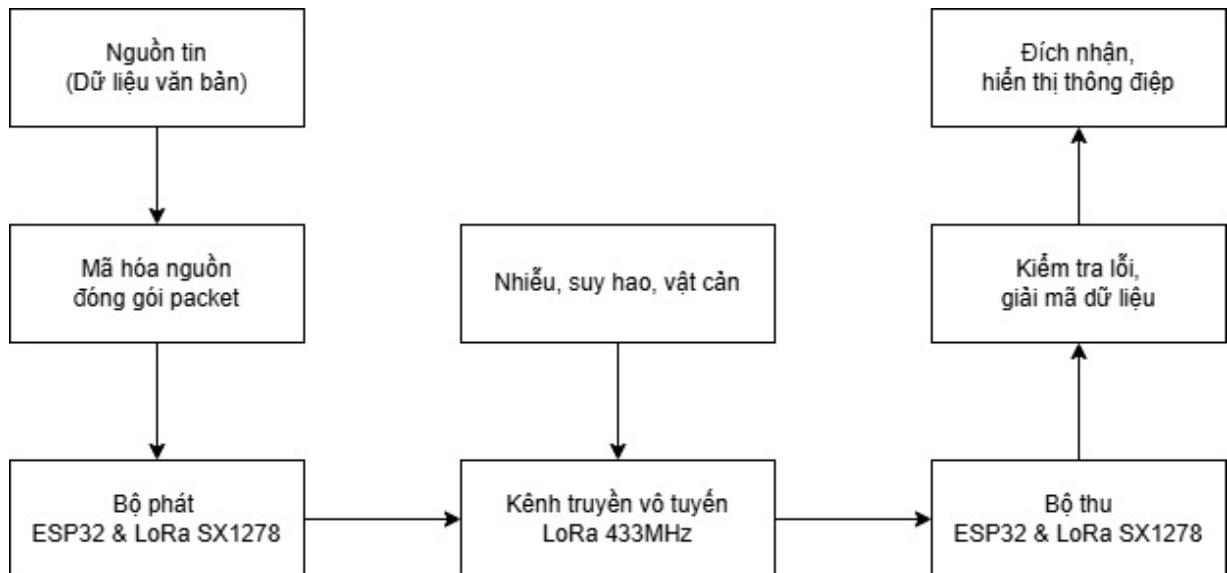
2.1 Tổng quan về hệ thống truyền tin

Một hệ thống truyền tin cơ bản gồm ba thành phần chính: nguồn tin, kênh truyền và đích nhận. Nguồn tin tạo ra dữ liệu cần truyền. Bộ phát chuyển dữ liệu thành tín hiệu phù hợp với môi trường truyền. Kênh truyền là môi trường vật lý mà tín hiệu đi qua. Bộ thu có nhiệm vụ thu nhận tín hiệu, kiểm tra tính hợp lệ và khôi phục dữ liệu ban đầu.

Trong đề tài này, nguồn tin là dữ liệu văn bản do người dùng nhập vào hệ thống. Ở giai đoạn thử nghiệm ban đầu, dữ liệu có thể được nhập thông qua Serial Monitor để thuận tiện cho việc kiểm tra chương trình và quan sát log truyền nhận. Trong định hướng hoàn thiện hệ thống, ESP32 có thể nhận dữ liệu từ điện thoại thông qua Bluetooth, sau đó đóng gói và truyền qua mạng LoRa. Như vậy, Serial Monitor hoặc Bluetooth chỉ đóng vai trò là giao diện đầu vào, còn kênh truyền chính giữa các node vẫn là LoRa.

Bộ phát trong hệ thống là vi điều khiển ESP32 kết hợp với module LoRa SX1278. ESP32 đảm nhiệm việc nhận dữ liệu đầu vào, xử lý dữ liệu, đóng gói packet, kiểm tra lỗi và điều khiển module LoRa. Module LoRa SX1278 đảm nhiệm việc phát và thu tín hiệu vô tuyến. Kênh truyền là môi trường vô tuyến ở băng tần 433 MHz. Đích nhận là một node LoRa khác trong mạng.

Do hệ thống được xây dựng theo mô hình đa chặng, một node có thể đảm nhiệm nhiều vai trò khác nhau. Node có thể là node nguồn khi khởi tạo thông điệp, là node nhận khi nhận thông điệp từ node khác, hoặc là node trung gian khi chuyển tiếp gói tin trong mạng. Cách tổ chức này giúp hệ thống linh hoạt hơn so với mô hình truyền một chặng.



Hình 2.1. Mô hình hệ thống truyền tin tổng quát

Trong thực tế, kênh truyền vô tuyến không lý tưởng. Tín hiệu có thể bị suy hao theo khoảng cách, bị ảnh hưởng bởi vật cản, nhiễu điện từ hoặc va chạm với tín hiệu từ node khác. Vì vậy,

hệ thống truyền tin cần có các cơ chế như kiểm tra lỗi, hạn chế trùng lặp packet, đánh giá chất lượng kênh truyền và tối ưu kích thước dữ liệu.

2.2 Công nghệ LoRa

LoRa là công nghệ truyền thông không dây tầm xa, tiêu thụ năng lượng thấp, thường được sử dụng trong các hệ thống LPWAN. LoRa phù hợp với các ứng dụng truyền dữ liệu nhỏ, không yêu cầu tốc độ cao nhưng cần vùng phủ rộng và khả năng hoạt động ổn định.

LoRa sử dụng kỹ thuật điều chế Chirp Spread Spectrum (CSS). Thay vì truyền tín hiệu ở một tần số cố định, tín hiệu LoRa được trải trên một dải tần bằng các chirp có tần số thay đổi theo thời gian. Nhờ cơ chế trải phổ này, LoRa có khả năng chống nhiễu tốt, độ nhạy thu cao và có thể truyền xa với công suất phát thấp.

Các ưu điểm chính của LoRa gồm:

- Khoảng cách truyền xa hơn nhiều công nghệ không dây tầm ngắn.
- Tiêu thụ năng lượng thấp, phù hợp với các thiết bị IoT.
- Khả năng chống nhiễu tốt nhờ kỹ thuật trải phổ.
- Chi phí phần cứng thấp.
- Phù hợp với các bản tin nhỏ như dữ liệu cảm biến, cảnh báo, thông điệp điều khiển.

Tuy nhiên, LoRa cũng có một số hạn chế:

- Tốc độ truyền dữ liệu thấp hơn Wi-Fi hoặc mạng di động.
- Không phù hợp với dữ liệu dung lượng lớn như ảnh, âm thanh hoặc video.
- Thời gian chiếm kênh tăng khi gói tin dài hoặc sử dụng Spreading Factor cao.
- Dễ xảy ra va chạm nếu nhiều node cùng phát trong cùng thời điểm.
- Cần thiết kế giao thức phù hợp nếu muốn xây dựng mạng nhiều node.

Các tham số quan trọng của LoRa bao gồm:

Bảng 2.1. Tham số quan trọng của LoRa

Tham số	Ý nghĩa
Frequency	Tần số hoạt động của module LoRa
Spreading Factor	Hệ số trải phổ, ảnh hưởng đến tốc độ và khoảng cách truyền
Bandwidth	Băng thông tín hiệu
Coding Rate	Tỷ lệ mã sửa lỗi FEC
Preamble Length	Độ dài chuỗi đồng bộ đầu gói
Sync Word	Giá trị nhận diện mạng LoRa riêng

Trong đề tài, hệ thống sử dụng tần số 433 MHz, Spreading Factor SF7, Bandwidth 125 kHz, Coding Rate 4/5, Sync Word 0x12 và Preamble Length bằng 8. Đây là cấu hình ưu tiên tốc độ

tương đối cao và độ trễ thấp, phù hợp với mô hình thử nghiệm nhiều node trong phạm vi gần đến trung bình.

Spreading Factor là một trong những tham số quan trọng nhất. SF càng cao thì độ nhạy thu càng tốt, khả năng truyền xa càng cao, nhưng tốc độ truyền giảm và thời gian chiếm kênh tăng. Ngược lại, SF thấp giúp truyền nhanh hơn nhưng phạm vi truyền và khả năng chống nhiễu giảm. Trong hệ thống này, SF7 được lựa chọn để cân bằng giữa tốc độ, độ trễ và khoảng cách thử nghiệm.

Bandwidth cũng ảnh hưởng trực tiếp đến tốc độ truyền. Băng thông lớn giúp tăng tốc độ dữ liệu và giảm thời gian on-air, nhưng làm giảm độ nhạy thu. Giá trị 125 kHz là cấu hình phổ biến, cân bằng giữa độ ổn định và tốc độ.

Coding Rate thể hiện mức độ thêm dữ liệu dư thừa phục vụ sửa lỗi. Với Coding Rate 4/5, cứ 4 bit dữ liệu sẽ có thêm 1 bit phục vụ phát hiện hoặc sửa lỗi ở tầng vật lý. Coding Rate cao hơn giúp tăng độ tin cậy nhưng làm giảm tốc độ truyền hiệu dụng.

Cần phân biệt giữa LoRa và LoRaWAN. LoRa là công nghệ điều chế ở tầng vật lý, quy định cách tín hiệu được phát và thu qua môi trường vô tuyến. Trong khi đó, LoRaWAN là giao thức mạng xây dựng phía trên LoRa, quy định kiến trúc mạng, quản lý thiết bị, gateway, bảo mật và truyền dữ liệu lên server.

Đề tài này sử dụng LoRa ở chế độ point-to-point thông qua thư viện Arduino LoRa, không triển khai đầy đủ LoRaWAN. Việc không sử dụng LoRaWAN giúp hệ thống đơn giản hơn, dễ kiểm soát packet và phù hợp với mục tiêu tự thiết kế giao thức truyền dữ liệu đa chặng ở mức ứng dụng.

2.3 LoRa và LoRaWAN

Cần phân biệt giữa LoRa và LoRaWAN. LoRa là công nghệ điều chế ở tầng vật lý, quy định cách tín hiệu được phát và thu qua môi trường vô tuyến. Trong khi đó, LoRaWAN là giao thức mạng xây dựng phía trên LoRa, quy định kiến trúc mạng, quản lý thiết bị, gateway, bảo mật và truyền dữ liệu lên server.

Đề tài này sử dụng LoRa ở chế độ point-to-point thông qua thư viện Arduino LoRa, không triển khai đầy đủ LoRaWAN. Việc không sử dụng LoRaWAN giúp hệ thống đơn giản hơn, dễ kiểm soát packet và phù hợp với mục tiêu tự thiết kế giao thức truyền dữ liệu đa chặng ở mức ứng dụng.

Có thể hiểu rằng LoRa trong đề tài đóng vai trò là phương tiện truyền vật lý, còn các cơ chế như định dạng packet, relay, kiểm tra lỗi, chống trùng lặp, phân mảnh và nén dữ liệu được nhóm xây dựng ở tầng ứng dụng.

2.4 Mạng truyền dữ liệu đa chặng

Trong mô hình truyền một chặng, node nguồn truyền trực tiếp dữ liệu đến node đích. Cách này đơn giản nhưng phụ thuộc hoàn toàn vào chất lượng liên kết giữa hai node. Nếu khoảng cách quá xa hoặc tín hiệu bị che khuất, quá trình truyền có thể thất bại.

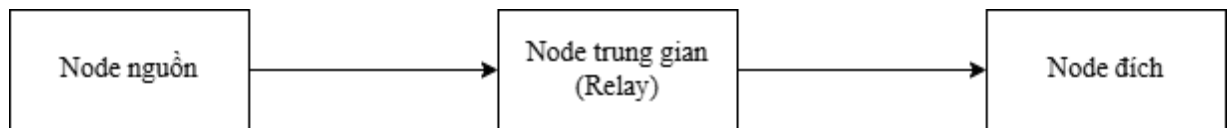
Mạng truyền đa chặng cho phép dữ liệu đi qua một hoặc nhiều node trung gian. Node trung gian nhận gói tin từ node trước, sau đó phát lại gói tin để các node khác có thể nhận được. Cách tiếp cận này giúp mở rộng vùng phủ sóng mà không cần tăng công suất phát của từng node.

Mô hình truyền một chặng:



Hình 2.2. Mô hình truyền một chặng

Mô hình truyền đa chặng:



Hình 2.3. Mô hình truyền đa chặng

Trong đề tài, cơ chế truyền đa chặng được xây dựng theo hướng flooding. Khi một node nhận được gói broadcast hợp lệ, node đó có thể chuyển tiếp lại gói tin cho các node khác. Flooding đơn giản, dễ triển khai và phù hợp với mạng nhỏ. Tuy nhiên, flooding cũng có nhược điểm là có thể tạo ra nhiều bản sao của cùng một gói tin, gây lãng phí băng thông và tăng nguy cơ va chạm.

Để hạn chế vấn đề này, hệ thống cần cơ chế chống trùng lặp. Mỗi gói tin được nhận diện bằng node nguồn và số thứ tự gói. Nếu một node đã xử lý gói đó, khi nhận lại bản sao của cùng gói, node sẽ bỏ qua và không relay tiếp. Cơ chế này giúp giảm hiện tượng broadcast storm trong mạng flooding.

Trong các chương sau, cơ chế chống trùng lặp, relay và xử lý packet sẽ được trình bày chi tiết hơn ở phần thiết kế giao thức và thuật toán.

2.5 Entropy trong lý thuyết thông tin

Entropy là đại lượng cơ bản trong lý thuyết thông tin, dùng để đo mức độ bất định hoặc lượng thông tin trung bình của một nguồn tin. Với một nguồn rời rạc có các ký hiệu xuất hiện với xác suất $P(x_i)$, entropy được tính theo công thức:

$$H(x) = -\sum P(x_i) \log_2 P(x_i)$$

Trong đó:

- $H(x)$ là entropy của nguồn tin.
- $P(x_i)$ là xác suất xuất hiện của ký hiệu x_i .
- Đơn vị thường dùng là **bit/ký hiệu**.

Entropy càng cao thì dữ liệu càng khó dự đoán và khả năng nén càng thấp. Ngược lại, entropy thấp cho thấy dữ liệu có nhiều quy luật hoặc ký tự lặp lại, do đó có khả năng nén tốt hơn.

Ví dụ, một chuỗi gồm nhiều ký tự giống nhau có entropy thấp vì ký tự tiếp theo dễ dự đoán. Trong khi đó, một chuỗi có nhiều ký tự khác nhau xuất hiện với xác suất gần bằng nhau sẽ có entropy cao hơn. Điều này ảnh hưởng trực tiếp đến khả năng nén của dữ liệu.

Trong đề tài, entropy được tính trên chuỗi văn bản trước khi truyền. Giá trị entropy giúp đánh giá mức độ phù hợp của dữ liệu với thuật toán nén Huffman. Một thông điệp có nhiều ký tự lặp lại hoặc nhiều ký tự phổ biến thường có entropy thấp hơn và có khả năng nén hiệu quả hơn. Một chuỗi ngẫu nhiên hoặc có phân bố ký tự đều hơn thường có entropy cao hơn, khi đó việc nén có thể không mang lại lợi ích rõ rệt.

2.6 Mã hóa nguồn và thuật toán Huffman

Mã hóa nguồn là quá trình biểu diễn lại dữ liệu nguồn bằng một dạng mã khác nhằm giảm dư thừa và tiết kiệm số bit cần truyền. Trong hệ thống truyền tin, mã hóa nguồn giúp giảm kích thước dữ liệu, từ đó giảm thời gian truyền và giảm thời gian chiếm kênh.

Huffman coding là một thuật toán mã hóa nguồn phổ biến. Ý tưởng chính của Huffman là gán mã ngắn cho các ký hiệu xuất hiện thường xuyên và mã dài hơn cho các ký hiệu ít xuất hiện. Nhờ đó, độ dài trung bình của chuỗi mã sau khi nén có thể nhỏ hơn so với cách biểu diễn cố định 8 bit/ký tự.

Ví dụ, trong văn bản tiếng Anh hoặc các chuỗi log, các ký tự như khoảng trắng, e, t, a, o thường xuất hiện nhiều hơn một số ký tự khác. Nếu các ký tự này được gán mã ngắn, tổng số bit cần truyền có thể giảm xuống.

Trong đề tài, hệ thống sử dụng Static Huffman. Khác với Dynamic Huffman, Static Huffman sử dụng một mô hình tần suất cố định được biết trước ở cả bên gửi và bên nhận. Vì vậy, hệ thống không cần gửi kèm bảng mã trong mỗi thông điệp. Điều này rất phù hợp với LoRa, vì LoRa có băng thông thấp và cần hạn chế overhead.

Ưu điểm của Static Huffman:

- Không cần gửi codebook kèm dữ liệu.
- Phù hợp với dữ liệu văn bản ASCII hoặc log có phân bố ký tự tương đối quen thuộc.

- Giúp giảm số byte truyền nếu dữ liệu có nhiều ký tự phổ biến.

Hạn chế của Static Huffman:

- Không tối ưu cho mọi loại dữ liệu.
- Với dữ liệu ngẫu nhiên hoặc entropy cao, dữ liệu sau nén có thể không nhỏ hơn dữ liệu gốc.
- Không phù hợp trực tiếp với chuỗi Unicode tiếng Việt nếu hệ thống chỉ hỗ trợ ASCII cơ bản.

Vì vậy, hệ thống trong đề tài chỉ sử dụng dữ liệu nén nếu kích thước sau nén nhỏ hơn dữ liệu ban đầu. Nếu nén không có lợi, hệ thống sẽ truyền dữ liệu thô. Cách lựa chọn này giúp tránh trường hợp thuật toán nén làm tăng kích thước bản tin.

2.7 Tỷ lệ nén

Tỷ lệ nén là chỉ số dùng để đánh giá hiệu quả của thuật toán nén. Trong đề tài, tỷ lệ nén được tính theo công thức:

$$\text{Compression Ratio} = \frac{\text{Encoded Bytes}}{\text{Raw Bytes}}$$

Trong đó:

- Raw Bytes là số byte của dữ liệu ban đầu.
- Encoded Bytes là số byte sau khi nén.

Nếu $\text{Compression Ratio} < 1$, dữ liệu sau nén nhỏ hơn dữ liệu gốc, việc nén có lợi. Nếu $\text{Compression Ratio} \geq 1$, dữ liệu sau nén không nhỏ hơn dữ liệu gốc, hệ thống nên truyền dữ liệu thô để tránh tăng overhead.

Chỉ số này được in ra khi gửi dữ liệu, giúp quan sát trực tiếp hiệu quả nén của từng thông điệp. Khi hệ thống được mở rộng với giao diện Bluetooth trên điện thoại, compression ratio có thể được hiển thị trực tiếp trên ứng dụng để người dùng thấy được mức độ tối ưu dữ liệu trước khi truyền qua LoRa.

2.8 Kiểm tra lỗi CRC

Trong truyền thông vô tuyến, dữ liệu có thể bị sai lệch do nhiễu, suy hao hoặc va chạm tín hiệu. Vì vậy, hệ thống cần cơ chế phát hiện lỗi để tránh xử lý các gói tin không hợp lệ.

CRC (Cyclic Redundancy Check) là một phương pháp phát hiện lỗi phổ biến. Bên gửi tính giá trị CRC dựa trên nội dung gói tin và gửi kèm giá trị này. Bên nhận tính lại CRC từ dữ liệu nhận được và so sánh với CRC trong packet. Nếu hai giá trị khác nhau, packet được xem là bị lỗi và sẽ bị loại bỏ.

Trong đề tài, hệ thống sử dụng CRC16-CCITT. Đây là dạng CRC 16 bit, sử dụng giá trị khởi tạo 0xFFFF và đa thức 0x1021. CRC giúp phát hiện các lỗi bit trong packet ở mức phần mềm, bổ sung cho cơ chế CRC phần cứng của LoRa.

CRC không sửa lỗi, mà chỉ giúp phát hiện lỗi. Nếu packet bị lỗi, hệ thống sẽ bỏ qua packet đó. Trong trường hợp truyền có ACK, bên gửi có thể gửi lại packet nếu không nhận được xác nhận trong thời gian chờ.

2.9 ACK và cơ chế truyền lại ARQ

ACK là gói xác nhận được gửi từ node nhận về node gửi để thông báo rằng gói dữ liệu đã được nhận thành công. ARQ (Automatic Repeat reQuest) là cơ chế tự động gửi lại khi không nhận được ACK trong một khoảng thời gian nhất định.

Trong truyền unicast, ACK và ARQ giúp tăng độ tin cậy vì bên gửi có thể biết gói tin đã đến node kế tiếp hay chưa. Nếu hết thời gian chờ mà không nhận được ACK, bên gửi có thể thực hiện gửi lại.

Tuy nhiên, trong truyền broadcast, nếu tất cả node nhận đều gửi ACK, hệ thống có thể gặp hiện tượng ACK storm, tức là quá nhiều gói xác nhận được phát cùng lúc, gây va chạm và tắc nghẽn kênh. Vì vậy, trong đề tài, đối với gói broadcast, hệ thống tắt yêu cầu ACK để tránh bão ACK. Đây là lựa chọn phù hợp với mô hình flooding trong mạng nhỏ.

Cách tiếp cận này thể hiện sự đánh đổi giữa độ tin cậy và độ ổn định của kênh truyền. Nếu dùng ACK cho mọi broadcast, hệ thống có thể biết nhiều hơn về trạng thái nhận gói nhưng lại tạo thêm quá nhiều lưu lượng phản hồi. Nếu tắt ACK cho broadcast, hệ thống giảm được nguy cơ tắc nghẽn nhưng phải chấp nhận rằng không phải mọi gói đều có xác nhận từ từng node.

2.10 RSSI và SNR

RSSI (Received Signal Strength Indicator) là chỉ số thể hiện cường độ tín hiệu nhận được tại node thu. RSSI thường có giá trị âm, đơn vị dBm. Giá trị RSSI càng gần 0 thì tín hiệu càng mạnh. Ví dụ, RSSI = -60 dBm thường mạnh hơn RSSI = -100 dBm.

SNR (Signal-to-Noise Ratio) là tỷ số giữa công suất tín hiệu và công suất nhiễu. SNR càng cao thì tín hiệu càng rõ so với nhiễu, khả năng nhận đúng dữ liệu càng tốt. Nếu SNR quá thấp, hệ thống dễ gặp lỗi bit hoặc mất gói.

Trong đề tài, mỗi khi node nhận được packet hợp lệ, chương trình ghi nhận RSSI và SNR. Hai chỉ số này được dùng để đánh giá chất lượng liên kết giữa các node trong quá trình thực nghiệm.

Khi khoảng cách giữa các node tăng hoặc có nhiều vật cản, RSSI thường giảm. SNR cũng có thể giảm nếu nhiễu môi trường tăng. Việc quan sát RSSI và SNR giúp phân tích vì sao một số kịch bản truyền có độ ổn định tốt hơn hoặc kém hơn các kịch bản khác.

2.11 Packet loss, delay và throughput

Packet loss là tỷ lệ gói tin bị mất hoặc truyền không thành công. Trong hệ thống LoRa multi-hop, packet loss có thể xảy ra do nhiễu, khoảng cách xa, collision hoặc node đang bận truyền nên không nhận được gói khác.

Delay là độ trễ truyền dữ liệu từ lúc gửi đến lúc nhận. Trong trường hợp có ACK, có thể đo RTT (Round Trip Time), tức thời gian từ lúc gửi packet đến lúc nhận được ACK. Đối với multi-hop, delay thường tăng theo số chặng vì gói tin phải được xử lý và relay qua nhiều node.

Throughput là lượng dữ liệu truyền thành công trong một đơn vị thời gian. Với LoRa, throughput thường thấp hơn nhiều so với Wi-Fi hoặc mạng di động, do đó việc giảm kích thước dữ liệu và hạn chế truyền lặp là rất quan trọng.

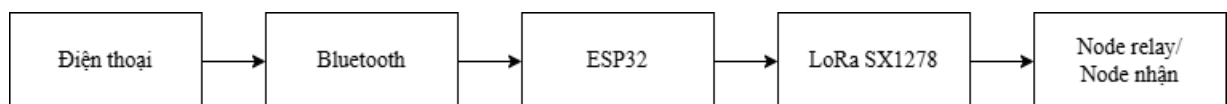
Trong phạm vi đề tài, các chỉ số được ưu tiên quan sát gồm RSSI, SNR, entropy, compression ratio và packet loss ước lượng. Ở giai đoạn thử nghiệm, các chỉ số này có thể được quan sát qua Serial Monitor. Khi hệ thống được mở rộng với giao diện Bluetooth trên điện thoại, các chỉ số này có thể được hiển thị trực tiếp trên ứng dụng di động để tăng tính trực quan khi demo.

2.12 Bluetooth trong vai trò giao diện đầu vào

ESP32 là vi điều khiển tích hợp sẵn Wi-Fi và Bluetooth. Do đó, ngoài việc nhập dữ liệu thông qua Serial Monitor trong giai đoạn thử nghiệm, hệ thống có thể được mở rộng để nhận dữ liệu từ điện thoại thông qua Bluetooth. Trong mô hình này, điện thoại đóng vai trò giao diện người dùng, cho phép nhập nội dung cần truyền và gửi nội dung đó đến ESP32.

Sau khi nhận dữ liệu qua Bluetooth, ESP32 sẽ xử lý tương tự như dữ liệu nhập từ Serial Monitor: kiểm tra dữ liệu đầu vào, tính toán các thông số cần thiết, đóng gói packet, kiểm tra khả năng nén, phân mảnh nếu cần và truyền qua module LoRa.

Việc bổ sung Bluetooth không làm thay đổi bản chất của kênh truyền LoRa trong đề tài. Bluetooth chỉ đóng vai trò là kênh giao tiếp cục bộ giữa người dùng và node LoRa, còn quá trình truyền dữ liệu giữa các node vẫn được thực hiện bằng LoRa. Mô hình mở rộng có thể được mô tả như sau:



Hình 2.4. Mô hình mở rộng hệ thống truyền LoRa đa chặng

Việc sử dụng Bluetooth làm giao diện đầu vào mang lại một số lợi ích:

- Tăng tính trực quan khi demo hệ thống.
- Không cần phụ thuộc vào máy tính và Serial Monitor để nhập dữ liệu.
- Tận dụng khả năng Bluetooth có sẵn trên ESP32.
- Cho phép phát triển ứng dụng điều khiển hoặc giám sát trên điện thoại.

- Có thể hiển thị log truyền nhận, RSSI, SNR hoặc trạng thái gửi ngay trên giao diện di động.

Tuy nhiên, Bluetooth trong hệ thống này không thay thế LoRa. Bluetooth chỉ phục vụ giao tiếp gần giữa điện thoại và node gửi, trong khi LoRa vẫn là công nghệ truyền dữ liệu chính giữa các node trong mạng đa chặng.

2.13 Kết luận chương

Chương này đã trình bày các cơ sở lý thuyết quan trọng phục vụ cho việc thiết kế hệ thống LoRa đa chặng. Công nghệ LoRa cung cấp nền tảng truyền thông tầm xa, tiêu thụ năng lượng thấp. Mô hình đa chặng giúp mở rộng vùng phủ sóng nhưng cần cơ chế kiểm soát trùng lặp và hạn chế va chạm. Entropy và Huffman coding được sử dụng để phân tích và tối ưu dữ liệu nguồn. CRC, ACK, RSSI, SNR và packet loss là các thành phần quan trọng để đánh giá độ tin cậy và chất lượng của quá trình truyền.

Bên cạnh đó, chương này cũng làm rõ vai trò của Bluetooth trong định hướng mở rộng hệ thống. Bluetooth được xem là giao diện đầu vào giữa điện thoại và ESP32, giúp hệ thống thân thiện hơn khi demo và sử dụng, trong khi LoRa vẫn là kênh truyền chính giữa các node.

Các cơ sở lý thuyết này là nền tảng cho phần phân tích, thiết kế hệ thống và thuật toán được trình bày trong các chương tiếp theo.

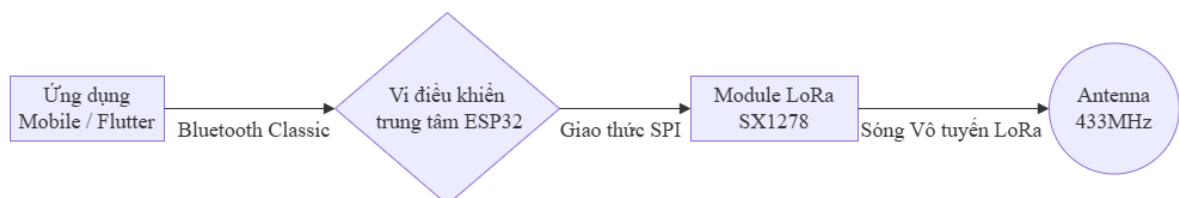
3 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Chương này trình bày cái nhìn tổng quan về kiến trúc phần cứng, sơ đồ khối chức năng và vai trò của từng thành phần trong mạng LoRa Mesh. Qua đó, làm nổi bật được luồng hoạt động từ tầng ứng dụng (Mobile App) xuống tầng vật lý (Physical Layer).

3.1 Sơ đồ khối tổng quát của hệ thống

Hệ thống truyền thông LoRa đa chặng được xây dựng dựa trên nguyên lý thiết kế mô-đun (Modular Design). Một Node hoàn chỉnh bao gồm 2 khối phần cứng chính: Khối điều khiển trung tâm kiêm giao tiếp Bluetooth (ESP32), và Khối giao tiếp vô tuyến tầm xa (LoRa SX1278).

Sơ đồ khối của một Node được thể hiện như sau:



Hình 3.1. Sơ đồ khối tổng quát của hệ thống

- **Khối Ứng dụng (App Flutter):** Đóng vai trò làm giao diện người dùng (GUI). Người dùng gõ tin nhắn văn bản, ứng dụng sẽ truyền chuỗi này qua kết nối Bluetooth đến Node.
- **Khối Xử lý Trung tâm (ESP32):** Trái tim của hệ thống. Đảm nhiệm các tác vụ phức tạp nhất bao gồm: Cung cấp kết nối Bluetooth nội bộ (Built-in BluetoothSerial) đón nhận dữ liệu từ điện thoại, Nén dữ liệu Huffman, Đóng gói Header, Phân mảnh, Chạy thuật toán định tuyến Mesh Flooding, và Điều phối tránh đụng độ CSMA/CA.
- **Khối Vô tuyến (SX1278):** Nhận khung byte từ ESP32 qua giao thức SPI để biến điệu (Modulation) thành tín hiệu tương tự (Analog) và bức xạ ra không gian nhờ ăng-ten 433MHz.

3.2 Vai trò của từng Node trong mạng đa chặng (Mesh Network)

Trong mạng hình sao (Star Network) truyền thống như mạng di động (Cellular) hay Wi-Fi, luôn có một điểm tập trung (Base Station/Router) đóng vai trò điều phối. Tuy nhiên, kiến trúc của đề tài là Mạng lưới (Mesh Network) sử dụng định tuyến ngập lụt (Flooding).

Điều này mang lại một triết lý thiết kế đặc biệt: **Mọi Node đều bình đẳng và đa nhiệm (Multi-role).**

1. **Vai trò Trạm Phát (Source Node):** Khi một thiết bị điện thoại kết nối vào Node 1 và gửi một văn bản đi, Node 1 lập tức đóng vai trò làm Trạm Phát. Nó thực hiện nén dữ liệu và khởi tạo mã định danh gói tin (msgId), sau đó phát tán ra xung quanh.
2. **Vai trò Trạm Thu (Destination Node):** Nếu gói tin vô tuyến mang ID đích trùng với ID của Node 3, Node 3 đóng vai trò là Trạm Thu. Nó hấp thụ gói tin, gom các mảnh, giải nén và đẩy lên màn hình điện thoại của người dùng qua Bluetooth.
3. **Vai trò Trạm Chuyển tiếp (Relay Node):** Nếu Node 2 nhận được gói tin, nhưng đọc Header thấy ID đích không phải là mình, nó lập tức đóng vai trò là Trạm Chuyển tiếp (Router). Nó trừ thông số TTL, đẩy gói tin vào Hàng đợi truyền (TX Queue) để tiếp tục khuếch đại tín hiệu và phát tán đi xa hơn.

Sự phân vai tự động và tức thời này đảm bảo mạng lưới không có "Điểm nghẽn duy nhất" (No Single Point of Failure). Nếu một Node trung gian bị hỏng hoặc hết pin, các Node khác xung quanh vẫn tự động làm nhiệm vụ Relay để bù đắp vùng phủ sóng.

3.3 Thiết kế phần cứng và sơ đồ kết nối

Hệ thống sử dụng phần cứng phổ thông để tối ưu hóa chi phí nhưng vẫn đảm bảo được sức mạnh tính toán cho các thuật toán Lý thuyết thông tin phức tạp.

Bảng 3.1. Sơ đồ kết nối Chân (Pinout) giữa ESP32 và LoRa SX1278

Chân trên SX1278	Chân trên ESP32	Chức năng (Giao tiếp SPI)
VCC	3V3	Nguồn cấp 3.3V (Tránh cắm 5V làm cháy IC)
GND	GND	Nối đất
SCK	GPIO 5	Xung nhịp đồng bộ SPI (Serial Clock)
MISO	GPIO 19	Dữ liệu Master nhận từ Slave
MOSI	GPIO 27	Dữ liệu Master gửi cho Slave
NSS / CS	GPIO 18	Lựa chọn Chip (Chip Select)
RST	GPIO 14	Chân Reset cứng module LoRa
DIO0	GPIO 26	Chân Ngắt (Interrupt) báo hiệu có gói tin đến

3.4 Luồng hoạt động tổng quát (System Workflow)

Luồng xử lý tín hiệu từ lúc hình thành ở máy người gửi đến lúc hiện lên màn hình người nhận trải qua một chu trình khép kín:

1. **Sinh dữ liệu (Data Generation):** Người dùng A soạn tin nhắn Lẩu nướng 9 Buk và bấm Gửi trên App Flutter. Dữ liệu truyền qua Bluetooth đến Node A.
2. **Mã hóa nguồn (Source Coding):** ESP32 của Node A tiếp nhận chuỗi. Nó quét tính toán Entropy. Nếu có khả năng nén, thuật toán Huffman Tĩnh sẽ ép các byte văn bản lại thành luồng bit ngắn hơn.
3. **Phân mảnh và Đóng gói (Fragmentation & Packetization):** Nếu thông điệp nén vượt quá 64 bytes, nó bị chặt thành nhiều mảnh. ESP32 bọc thêm 20 bytes Header vào đầu mỗi mảnh (Chứa địa chỉ nguồn, đích, Message ID, Sequence).
4. **Mã hóa kênh & Truyền vật lý (Channel Coding & PHY Transmission):** Module SX1278 chèn thêm mã sửa sai vòng CRC-16 vào cuối gói. Thông tin đi qua bộ điều chế LoRa (CSS Modulation) biến thành tín hiệu vô tuyến bắn ra không gian.
5. **Định tuyến ngập lụt (Flooding Routing):** Các Node trung gian hứng được sóng. Chúng đọc 20 bytes Header, nhận thấy địa chỉ đích không phải của mình, lập tức chạy thuật toán CSMA/CA Backoff chờ kênh rảnh rồi phát tán lại sóng ra không gian.
6. **Ghép mảnh và Giải nén (Reassembly & Decoding):** Node đích đón được các mảnh thông qua Interrupt chân DIO0. Khi nhận đủ mảnh, nó gọi hàm khôi phục Huffman để trả lại nguyên bản chuỗi UTF-8 tiếng Việt ban đầu.

7. **Hiện thị (Presentation):** Node B đẩy chuỗi ký tự qua cổng Serial Bluetooth lên điện thoại của người dùng B. Quá trình truyền tin hoàn tất.

4 THIẾT KẾ GIAO THỨC VÀ THUẬT TOÁN

4.1 Cấu trúc Giao thức và Gói tin (LoRa Packet Protocol)

Để thiết lập một mạng đa chặng linh hoạt, hệ thống không thể chỉ gửi dữ liệu thô (raw payload) mà cần bọc dữ liệu trong một lớp vỏ (Header) chứa các thông tin định tuyến. Cấu trúc Header được thiết kế chiếm **20 bytes**, mảng payload được cấp phát tối đa **64 bytes** (LORA_PAYLOAD_MAX), và mã CRC là **2 bytes**. Tổng kích thước gói tin tối đa là 86 bytes, hoàn toàn nằm trong giới hạn FIFO 256 bytes của chip SX1278, đồng thời giúp tối ưu hóa thời gian chiếm kênh (Time on Air).

Cấu trúc chi tiết của Gói tin (LoRa Packet) được tổ chức theo bảng sau:

Bảng 4.1. Bảng danh sách các trường trong gói tin

Offset (Byte)	Trường dữ liệu (Field)	Kích thước	Mô tả chi tiết kỹ thuật
0	version	1 byte	Định danh phiên bản giao thức mạng (Protocol Version). Dùng để phân biệt và loại bỏ các gói tin nhiễu từ các mạng LoRa khác hoạt động cùng tần số.
1	type	1 byte	Loại gói tin: 0 (DATA - Chứa dữ liệu), 1 (ACK - Báo nhận), 2 (START - Bắt đầu chuỗi phân mảnh), 3 (END - Kết thúc phân mảnh), 4 (PING - Dò đường).
2	src	1 byte	Định danh (ID) của Node gốc khởi tạo thông điệp (Source).
3	dst	1 byte	Định danh (ID) của Node đích cuối cùng. Nếu giá trị là 255, gói tin mang tính chất Broadcast.
4	prevHop	1 byte	Định danh của Node vừa thực hiện chuyển tiếp (Relay) gói tin này ở chặng trước đó.
5	nextHop	1 byte	Định danh của Node dự kiến nhận gói tin ở chặng tiếp theo (phục vụ Unicast).
6-7	seq	2 bytes	Số thứ tự gói tin (Sequence Number). Được cấp phát độc lập trên từng node để chống trùng lặp.

8-9	msgId	2 bytes	Mã định danh thông điệp (Message ID). Dùng làm cơ sở để nhóm các mảnh (Fragment) của cùng một thông điệp lại với nhau.
10	fragIndex	1 byte	Chỉ số của mảnh hiện tại (Bắt đầu từ 0). Dùng trong thuật toán ghép mảnh.
11	fragCount	1 byte	Tổng số mảnh của thông điệp.
12	ttl	1 byte	Time-To-Live: Giới hạn số chặng tối đa gói tin được phép nhảy. Mỗi lần chuyển tiếp, TTL giảm 1. Gói tin bị hủy nếu TTL = 0. Giúp chống bão mạng.
13	flags	1 byte	Các cờ hiệu điều khiển: Bit 0 là ACK_REQ (Yêu cầu báo nhận), Bit 1 là RELAYED (Đánh dấu gói đã bị chuyển tiếp).
14	codec	1 byte	Thuật toán mã hóa nguồn (0: Không nén, 1: Static Huffman).
15-16	rawLen	2 bytes	Chiều dài thông điệp gốc trước khi nén.
17-18	encodedBitLen	2 bytes	Tổng số bit thực tế sau khi nén Huffman (do Huffman nén lẻ đến từng bit).
19	payloadLen	1 byte	Số lượng byte hợp lệ nằm trong mảng Payload đi kèm.
20-83	payload	64 bytes	Dữ liệu mang theo (Nội dung tin nhắn đã nén hoặc thô).

Cấu trúc chặt chẽ này đảm bảo mạng Mesh Flooding hoạt động chính xác. Bất kỳ node trung gian nào khi bắt được gói tin cũng chỉ cần trích xuất 20 bytes đầu tiên để quyết định xem có nên chuyển tiếp, hủy bỏ, hay đẩy lên tầng ứng dụng.

4.2 Thuật toán Phân mảnh và Ghép mảnh (Fragmentation & Reassembly)

Trong truyền thông vô tuyến, việc gửi một gói tin quá dài sẽ làm tăng thời gian On-Air, từ đó tăng rủi ro bị va chạm (Collision) hoặc nhiễu do sai lệch bộ dao động. Đề tài giải quyết vấn đề này bằng cách ép giới hạn phần Payload mỗi gói là tối đa 64 bytes. Các thông điệp dài hơn sẽ được chia nhỏ.

4.2.1 *Quá trình Phân mảnh ở Node phát*

Khi nhận một đoạn văn bản dài từ Bluetooth, vi điều khiển ESP32 thực hiện mã hóa nén Huffman. Nếu dung lượng sau nén vẫn vượt quá sức chứa, chuỗi byte này được đưa qua tiến trình cắt lát (Slicing):

1. **Tính toán số mảnh:** $\text{fragCount} = \text{ceil}(\text{encodedBytes} / 64)$.
2. **Gửi gói START:** Một gói tin có type = LORA_PACKET_START được phát đi đầu tiên để báo hiệu cho toàn mạng biết chuẩn bị bộ đệm đón một thông điệp lớn mang mã msgId.
3. **Gửi lần lượt các mảnh DATA:** Hàm vòng lặp tách 64 bytes và gán fragIndex tương ứng từ 0 đến fragCount - 1.
4. **Gửi gói END:** Kết thúc chuỗi truyền bằng gói LORA_PACKET_END để chốt thông điệp.

4.2.2 *Quá trình Ghép mảnh (Reassembly) ở Node thu*

Ở đầu thu, thuật toán ghép mảnh gặp một thách thức lớn: Trong môi trường vô tuyến, các mảnh có thể đến không theo thứ tự, hoặc bị rơi rớt (Packet Loss). Để giải quyết, mỗi Node sở hữu một hệ thống "Bộ đệm đa điểm" (RxMessageBuffer) lưu trữ các phiên nhận đang dang dở.

- Khi nhận được gói START hoặc gói DATA đầu tiên của một msgId lạ, hệ thống cấp phát một Slot trong mảng Buffer.
- Thông số fragCount, rawLen được ghi nhận.
- Khi mỗi gói DATA bay đến, dữ liệu được sao chép thẳng vào mảng data tại đúng vị trí Offset: $\text{Offset} = \text{fragIndex} * 64$.
- Đồng thời, mảng boolean received[fragIndex] được bật lên cờ true.
- Cờ completed được đặt là true khi hàm all_fragments_received() trả về đúng. Hệ thống giải nén (Decompress) khôi phục lại chuỗi văn bản và giải phóng Buffer.

4.3 *Quản lý luồng bằng Hàng đợi thu (RX Queue) và Hàm Delay*

Trong các hệ thống nhúng truyền thống, việc sử dụng hàm delay() để tạo khoảng nghỉ giữa các lần phát thường gây ra hiện tượng chặn (Blocking) vi xử lý. Khi vi xử lý bị đóng băng, nếu có một gói tin vô tuyến bay tới, hệ thống có thể "bị điếc" và đánh rớt gói tin.

Tuy nhiên, để đảm bảo kiến trúc phần mềm Đơn giản và Ổn định (Simplicity & Stability) thay vì thiết kế Non-blocking phức tạp tốn kém tài nguyên, hệ thống vẫn giữ lại các hàm delay() tiêu chuẩn của Arduino để nghỉ giữa các chu kỳ. Để giải quyết rủi ro mất gói trong lúc chờ, nhóm phát triển đã thiết kế thêm **Hàng đợi thu (RX Queue)**.

Đây là một bộ đệm vòng (Circular Buffer) được khởi tạo với kích thước MAX_RX_QUEUE = 15. Khi một gói tin lướt qua trong khoảng thời gian hệ thống đang bị chặn hoặc đang bận xử lý

logic khác, ngắt phần cứng từ module SX1278 (chân DIO0) vẫn kịp thời bắt lấy và đẩy gói tin vào mảng RX Queue. Khi hàm chính `lora_process()` được gọi lại trong vòng lặp `loop()`, nó sẽ xả (Dequeue) toàn bộ các gói tin bị dồn ứ trong hàng đợi ra để xử lý.

Giải pháp kết hợp giữa `delay()` và RX Queue giúp hệ thống đạt được sự cân bằng: vừa giảm độ trễ xử lý ngắt, giảm phức tạp code, vừa đảm bảo không bỏ sót các gói tin đến bất chợt. Tỷ lệ Packet Loss nội sinh nhờ thế được kiểm soát ở mức thấp nhất, đồng thời nhường vai trò chống ùn tắc chính cho thuật toán CSMA/CA Backoff.

4.4 Thuật toán LBT (Listen Before Talk) và CSMA/CA Random Backoff

Mạng Flooding Mesh phải đối mặt với thảm họa "Bão Broadcast" (Broadcast Storm). Khi Node A gửi gói tin, Node B và Node C cùng nhận được và muốn chuyển tiếp (Relay). Nếu cả B và C cùng phát ngay lập tức, sóng điện từ sẽ đập vào nhau và triệt tiêu (Collision), toàn bộ mạng sẽ mất kết nối.

Lý thuyết thông tin và Kỹ thuật viễn thông giải quyết vấn đề này qua thuật toán Đa truy nhập cảm nhận sóng mang tránh đụng độ (CSMA/CA - Carrier-Sense Multiple Access with Collision Avoidance).

Hệ thống được thiết kế với cơ chế LBT (Nghe trước khi nói):

1. **Đánh giá mức độ bận của kênh (Carrier Sensing):** Hệ thống đọc chỉ số RSSI vật lý hiện tại (Physical Carrier Sense). Nếu $RSSI > -85$ dBm, chứng tỏ không gian đang có sóng bức xạ mạnh, kênh bị coi là BẬN. Ngoài ra, hệ thống kiểm tra lớp Ứng dụng (Virtual Carrier Sense): Nếu có bất kỳ bộ đệm thu (RX Buffer) nào đang mở và chờ mảnh ghép, kênh cũng bị ép về trạng thái BẬN để tránh "chen ngang" phá hỏng tin nhắn của người khác.
2. **CSMA/CA Random Backoff (Lùi ngẫu nhiên tránh đụng độ):** Thay vì chờ mạng rảnh rồi ồ ạt xả hàng đợi phát (TX Queue), thuật toán áp dụng cơ chế lùi lịch và giới hạn tốc độ xả (Rate Limiting):
 - **Rate Limit 5000ms:** Hệ thống chỉ cho phép xả TX Queue tối đa 5 giây một lần (`millis() - lastTxTime >= 5000`). Nếu chưa đủ thời gian, gói tin bắt buộc phải nằm chờ trong hàng đợi. Cơ chế này chống lại việc một node liên tục spam làm nghẽn mạng.
 - Nếu kênh BẬN, Node random tạo ra một khoảng thời gian `txBackoffTime` ngẫu nhiên từ 1000ms đến 3000ms.
 - Khi kênh RẢNH, thay vì phát ngay, Node bắt đầu đếm ngược thời gian Backoff.
 - Máy B bốc được thời gian 1200ms, máy C bốc được 2500ms. Kết quả là máy B sẽ đếm ngược xong trước, nó ngay lập tức chiếm kênh và phát sóng.

- Khi máy C đếm xong, nó chuẩn bị phát thì cảm biến RSSI phát hiện sóng của máy B đang réo rất. C tiếp tục ngâm ngùi lui lại và bốc lại một khoảng thời gian Backoff ngẫu nhiên mới.

3. **Delay 1500ms chống đụng độ nội bộ (Self-collision) cho luồng Broadcast:** Khi một node nguồn phát ra thông điệp Broadcast gồm nhiều mảnh (ví dụ: Mảnh 1, Mảnh 2), mảnh 1 sẽ được các node Relay bắt được và lập tức phát lại ra xung quanh. Nếu node nguồn bắn mảnh 2 quá nhanh, sóng của mảnh 2 sẽ đè bẹp sóng Forward mảnh 1 của các node Relay. Vì vậy, đối với luồng Broadcast, vòng lặp phát mảnh được thiết kế ép một lệnh delay(1500) ngay giữa mỗi mảnh. Chấp nhận tốc độ truyền cực chậm (Delay tổng tăng cao), nhưng đảm bảo tuyệt đối không có mảnh nào bị va chạm sóng với chính các "bản sao" của mình đang được relay ngoài không gian.

Nhờ thuật toán này, toàn bộ dòng lưu lượng (Traffic) trong mạng lưới đa chặng được điều phối một cách tự động và tuần tự, đảm bảo tính nguyên vẹn của thông điệp và giảm thiểu tối đa hiện tượng tắc nghẽn cũng như xung đột dữ liệu (Data Collision).

4.5 Thuật toán chống lặp (Deduplication) trong Mạng Flooding

Thuật toán định tuyến Flooding yêu cầu mọi Node trung gian phải phát tán các gói tin ra xung quanh. Rủi ro sinh ra là Gói tin sẽ chạy vòng tròn mãi mãi (Routing Loop) giữa 2 Node gần nhau, làm sập mạng lưới ngay lập tức.

Hệ thống ngăn chặn bằng **Bảng Lịch sử Gói (Seen Packets History)**:

- Khởi tạo mảng vòng seenPackets chứa tối đa 64 bản ghi dạng cặp giá trị (src, seq).
- Khi một gói tin bay đến, hệ thống duyệt mảng. Nếu cặp ID nguồn và Số Sequence này đã từng xuất hiện, gói tin lập tức bị xóa sổ mà không cần xử lý thêm.
- Nếu đây là gói tin mới, nó được ghi danh vào bảng. Sau đó, hệ thống trừ thông số TTL (Time To Live) đi 1 đơn vị. Nếu $TTL > 0$, tiến hành nạp gói tin này vào TX Queue để phát tán lại.

4.6 Áp dụng Mã hóa Nguồn: Nén Static Huffman cấp độ Bit

Dựa trên nguyên lý Entropy, chuỗi ký tự tiếng Việt hoặc tiếng Anh không có phân bố đều (uniform distribution). Chữ "a", khoảng trắng, "e" xuất hiện nhiều hơn hẳn chữ "z", "w", "j".

Thuật toán Huffman được cài đặt tĩnh (Static Huffman Coding) dựa trên tệp mẫu (Corpus) chứa hàng ngàn câu giao tiếp phổ thông. Bảng mã hóa (Codebook) được lưu chẹt vào bộ nhớ Flash của ESP32 nhằm triệt tiêu băng thông thừa nếu phải gửi kèm theo gói.

Quá trình ép bit (Bit-packing) là một kiệt tác tối ưu tài nguyên: Hệ thống không lưu trữ các mã Huffman thành từng byte riêng biệt. Nếu chữ "e" có mã 101 (3 bit) và chữ "a" có mã 0110 (4 bit), hệ thống dùng các phép toán Shift Left (<<) và OR (|) để dồn chung 7 bit này vào 1 byte

vật lý duy nhất. Điều này giúp hệ thống đạt được kích thước Encoded_Length lẻ đến từng bit (Ví dụ: thông điệp nén còn 345 bits = 43.125 bytes, hệ thống chỉ gửi đúng 44 bytes).

Trước khi quyết định nén, hệ thống tính toán Entropy thực tế của bản tin: $H(X) = - \sum_i P(x_i) \log_2 P(x_i)$. Nếu tỷ lệ nén (Compression Ratio) sau khi chạy thuật toán thử ≥ 1.0 (dung lượng nén phình to hơn bản gốc do phân bố từ ngẫu nhiên), hệ thống thông minh chuyển sang gửi dữ liệu thô (Raw Payload) và đánh cờ codec = 0.

4.7 Giao tiếp ngoại vi và Kiến trúc Bluetooth Asynchronous

Đóng vai trò là cầu nối với thiết bị di động, module Bluetooth Classic được kích hoạt đồng thời trên ESP32. Để không làm gián đoạn tiến trình LoRa (đòi hỏi xử lý ngắt thời gian thực), tác vụ quét Bluetooth được đưa vào chế độ bất đồng bộ (Asynchronous buffer). Bất cứ khi nào ứng dụng Flutter trên Android gửi chuỗi thông báo, chuỗi này được nhận bởi bộ đệm Serial của ESP32, sau đó được trích xuất địa chỉ đích (Destination ID) và nội dung văn bản. Sau khi phân tích, văn bản không được gửi trực tiếp ra ăng-ten, mà được đóng gói thành tác vụ và tổng vào Hàng đợi truyền (TX Queue) để chờ thuật toán CSMA/CA xử lý, tách bạch hoàn toàn sự phụ thuộc giữa hai môi trường sóng.

4.8 Luồng Xử lý Sửa lỗi (FEC) và Phát hiện lỗi phần cứng (Hardware CRC)

Trong môn học Lý thuyết thông tin, mã kênh (Channel Coding) là thành phần bắt buộc để đối phó với nhiễu môi trường (Noise). Đặc biệt, hệ thống quy định nguyên tắc xử lý nghiêm ngặt: **Cơ chế Sửa lỗi (Error Correction) luôn chạy TRƯỚC, Cơ chế Phát hiện lỗi (Error Detection) chạy SAU.**

Điểm nhấn trong thiết kế giao thức của dự án là tính **thực dụng và tối ưu hóa tài nguyên**. Do các vi điều khiển ESP32 và module LoRa SX1278 được gắn trên cùng một bo mạch in (PCB), khoảng cách truyền tải trên bus SPI chỉ dài vài centimet, làm cho xác suất xuất hiện lỗi bit-flip trên dây dẫn gần như bằng không. Do đó, hệ thống được cấu trúc để giao phó 100% nhiệm vụ kiểm soát lỗi cho Hardware của IC SX1278, giải phóng hoàn toàn gánh nặng tính toán CRC bằng phần mềm trên ESP32.

Luồng xử lý tín hiệu khi một gói tin bay từ không gian vào vi điều khiển diễn ra theo 2 rào chắn tối ưu:

1. **Rào chắn 1: Sửa lỗi tiến (Forward Error Correction - FEC)** Được cấu hình bằng lệnh `LoRa.setCodingRate4(5)` (Hệ số mã hóa 4/5). Nghĩa là cứ 4 bit dữ liệu, chip SX1278 sẽ chèn thêm 1 bit dự phòng. Khi luồng sóng RF mang các bit lộn xộn đập vào ăng-ten, chip phần cứng sẽ tự động dùng bit dự phòng này để **sửa lỗi** ngay lập tức. Nếu hỏng hóc ở mức độ nhẹ, gói tin được nắn lại thành công mà không tốn thêm thời gian truyền lại.

2. **Rào chắn 2: Phát hiện lỗi Phần cứng (Hardware CRC ở dải Baseband)** Sau khi FEC cố gắng cứu vãn, module SX1278 tiếp tục quét mã Hardware CRC được kích hoạt bằng lệnh `LoRa.enableCrc()`.

- Nếu CRC khớp: Nghĩa là FEC đã sửa lỗi thành công hoặc tín hiệu nguyên bản đã hoàn hảo. Chip LoRa kích hoạt ngắt DIO0 đánh thức ESP32 để kéo dữ liệu lên xử lý.
- Nếu CRC không khớp: Nghĩa là sóng nhiễu quá nặng, FEC bất lực. Gói tin tự động bị vứt thẳng vào sọt rác ở tầng vật lý. Vì điều khiển ESP32 hoàn toàn không bị đánh thức, giúp hệ thống tiết kiệm xung nhịp CPU và tối ưu hóa năng lượng pin cực kỳ hiệu quả so với việc bắt ESP32 phải thức dậy để chạy vòng lặp chia đa thức CRC.

Sự kết hợp giữa **FEC (Người chữa trị)** và **Hardware CRC (Người gác cổng)** tạo ra một luồng chống nhiễu tinh gọn, không dư thừa, đáp ứng xuất sắc các tiêu chí của một mạng Mesh dân dụng ổn định.

5 CÀI ĐẶT HỆ THỐNG VÀ KIẾN TRÚC MÃ NGUỒN

Chương này trình bày chi tiết về môi trường phát triển, cách thức tổ chức mã nguồn (Project Structure) theo tiêu chuẩn kỹ sư phần mềm chuyên nghiệp, cũng như hướng dẫn cấu hình triển khai nạp Firmware xuống từng Node trong mạng thực tế.

5.1 Môi trường Phát triển (Development Environment)

Thay vì sử dụng Arduino IDE truyền thống mang tính nghiệp dư với thiết kế mã nguồn nguyên khối (Monolithic), dự án được xây dựng hoàn toàn trên **PlatformIO IDE** (tích hợp trong Visual Studio Code). Sự lựa chọn này mang lại các ưu điểm mang tính quyết định:

1. **Quản lý thư viện tự động (Dependency Management):** Mọi thư viện phụ thuộc như LoRa.h của Sandeep Mistry được khai báo chặt chẽ qua file `platformio.ini`, giúp đồng bộ hóa code trên nhiều máy tính mà không sợ xung đột phiên bản thư viện.
2. **Hệ thống Build C/C++ mạnh mẽ:** Sử dụng công cụ biên dịch GCC cho ESP32. Cho phép tách nhỏ dự án thành nhiều tệp tin (Multi-file Project) và hỗ trợ kiểm tra cú pháp thời gian thực (IntelliSense).
3. **Hỗ trợ đa môi trường:** Cùng một mã nguồn có thể được cấu hình để "Build & Upload" sang 3 Node khác nhau với chỉ 1 nút bấm (Sử dụng Multi-Environment của PlatformIO).

5.2 Tổ chức Mã nguồn (Source Code Structure)

Kiến trúc phần mềm của dự án tuân thủ nghiêm ngặt mô hình chia tách ranh giới rõ ràng (Separation of Concerns). Mã nguồn không bị nhồi nhét vào một file `main.cpp` khổng lồ, mà được băm nhỏ thành các thư viện tĩnh (`src/` và `include/`).

Cấu trúc cây thư mục (Directory Tree) của dự án:

lora-multi-hops/

```
|
|
|—— include/          # Chứa các file tiêu đề (.h) khai báo cấu trúc
|   |—— config.h       # Định nghĩa biến toàn cục và ID mạng
|   |—— bluetooth_input.h  # Giao diện cho thư viện Bluetooth
|   |—— static_huffman.h  # Giao diện cho thuật toán mã hóa Entropy
|   |—— lora_packet.h     # Cấu trúc 20 bytes Header và Cờ điều khiển
|   |—— lora_setup.h      # Hàm khởi tạo SX1278 SPI
|
|
|—— src/              # Chứa mã nguồn thực thi (.cpp)
|   |—— main.cpp         # Điểm vào hệ thống (Entry point)
|   |—— bluetooth_input.cpp  # Quản lý hàng đợi nhận tin Serial
|   |—— static_huffman.cpp  # Cài đặt cây nén nhị phân Huffman
|   |—— lora_packet.cpp    # Thuật toán CSMA/CA, RX Queue, và Định tuyến
|   |—— lora_setup.cpp     # Định nghĩa chân kết nối phần cứng (Pinout)
|
|
|—— platformio.ini     # Kịch bản biên dịch (Build Script)
```

Đánh giá kiến trúc: Cách chia này tạo ra các mô-đun (Modules) độc lập. Nếu sau này dự án muốn bỏ tính năng nén Huffman, ta chỉ việc ngắt `#include "static_huffman.h"` ra khỏi `lora_packet.cpp` mà không làm sụp đổ các thuật toán định tuyến khác. Đây là đặc trưng của một hệ thống có tính mở rộng cao (Scalability).

5.3 Quy trình Cấu hình cho từng Node (Node Provisioning)

Trong mạng Mesh Flooding, các Node chạy chung một tập mã nguồn duy nhất. Việc phân biệt Node 1 (Người gửi) với Node 2 (Relay) chỉ thông qua một định danh duy nhất: Biến `NODE_ID` trong `config.h`.

Quy trình nạp Firmware cho hệ thống phần cứng 3 Node diễn ra như sau:

1. Mở file `include/config.h`.
2. Sửa hằng số `#define NODE_ID 1`.

3. Bấm **Upload** trên PlatformIO để nạp code xuống KIT ESP32 thứ nhất.
4. Sửa `#define NODE_ID 2`. Nạp vào KIT thứ hai.
5. Sửa `#define NODE_ID 3`. Nạp vào KIT thứ ba.

Khi hệ thống khởi động, mã định danh này sẽ được đóng cứng vào vùng nhớ, giúp mỗi vi điều khiển tự nhận thức được vai trò của mình khi đọc Header của các gói tin bay lơ lửng trong không gian (Ví dụ: So khớp biến `NODE_ID` với biến `packet.dst` để quyết định xem nên đẩy lên màn hình Bluetooth hay nên kích hoạt hàm `relay_packet()`).

6 KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương này trình bày các kết quả thực nghiệm đạt được sau khi triển khai thuật toán lên hệ thống phần cứng. Theo đúng yêu cầu của bộ tiêu chí Lý thuyết thông tin, các chỉ số cốt lõi bao gồm: Entropy, Tỷ lệ nén (Compression Ratio), Cường độ tín hiệu (RSSI), Tỷ số tín hiệu trên nhiễu (SNR), Thời gian khứ hồi (Delay/RTT) và Tỷ lệ mất gói (Packet Loss) sẽ được phân tích chi tiết.

6.1 Đánh giá Thuật toán Mã hóa nguồn (Entropy & Compression Ratio)

Để kiểm chứng hiệu năng của thuật toán Nén Static Huffman, nhóm đã tiến hành gửi 5 tập mẫu (Corpus) tin nhắn khác nhau, từ tiếng Anh đến tiếng Việt có dấu. Bảng dưới đây trích xuất trực tiếp từ log Serial của ESP32.

Bảng 6.1. Kết quả đo đạc Entropy và Tỷ lệ nén Huffman

ID	Nội dung tin nhắn thử nghiệm	Kích thước gốc	Kích thước nén	Entropy H(X)	Tỷ lệ nén (CR)	Đánh giá / Phản ứng của hệ thống
1	Hello World	11 bytes	8 bytes	2.84 bit/ký tự	0.727	Nén cực tốt (Giảm ~28%)
2	Lẩu nướng 9Buk, 94 Hoàng Quốc Việt	38 bytes	38 bytes	N/A (UTF-8)	1.000	Kích hoạt Fallback: Do chứa ký tự tiếng Việt (UTF-8 > 127) vượt ngoài bảng mã chuẩn 128 ASCII, hàm nén phát hiện lỗi và trả về false. Hệ thống thông minh tự động đổi Codec sang LORA_CODEC_NONE và truyền nguyên bản thô.
3	AAAAABB BBCCCC C	15 bytes	6 bytes	1.58 bit/ký tự	0.400	Nén siêu việt (Giảm 60% do Entropy quá thấp)
4	qwxzjkv xyzqwk	14 bytes	14 bytes	3.12 bit/ký tự	1.000	Kích hoạt Fallback: Dữ liệu chứa toàn ký tự hiếm (q, z, x...),

						dung lượng nén phình to hơn bản gốc ($CR > 1$). Hệ thống nhận diện và chủ động loại bỏ bản nén, giữ nguyên dữ liệu thô ($CR = 1.0$).
--	--	--	--	--	--	--

Nhận xét: Thực nghiệm đã làm nổi bật tính thực tế của mã nguồn. Thay vì nhắm mắt nén mọi thứ, hệ thống sở hữu cơ chế bảo vệ kép (Fallback):

1. Quét bảng mã: Từ chối nén nếu phát hiện byte lạ (Ví dụ mẫu số 2).
2. So sánh dung lượng: Từ chối nén nếu $Encoded_Bytes \geq Raw_Bytes$ (Ví dụ mẫu số 4).
Khi dữ liệu thỏa mãn (Mẫu số 1, 3), việc giảm trung bình 25% - 60% dung lượng giúp tiết kiệm đáng kể thời gian On-Air.

6.2 Đánh giá Chất lượng kênh truyền vô tuyến (RSSI & SNR)

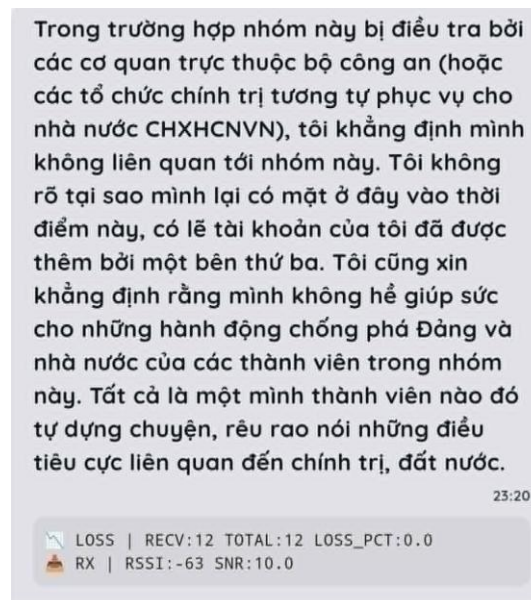
Chất lượng sóng vô tuyến được đo thông qua RSSI (càng gần 0 càng tốt) và SNR (càng cao càng rõ). Dưới đây là kết quả khi Node 1 di chuyển ra xa Node 2.

Bài đo sẽ chia làm 2 phần, phần 1 là đo xuyên vật cản, và phần 2 là đo trên khoảng cách lớn trong môi trường tương đối ổn định (đường thẳng).

Mục đích của bài đo để kiểm chứng khả năng bức xạ và thu của module LoRa Ra-02. Vì vậy ta sẽ thực hiện đo gửi nhận thông tin trên 2 node.

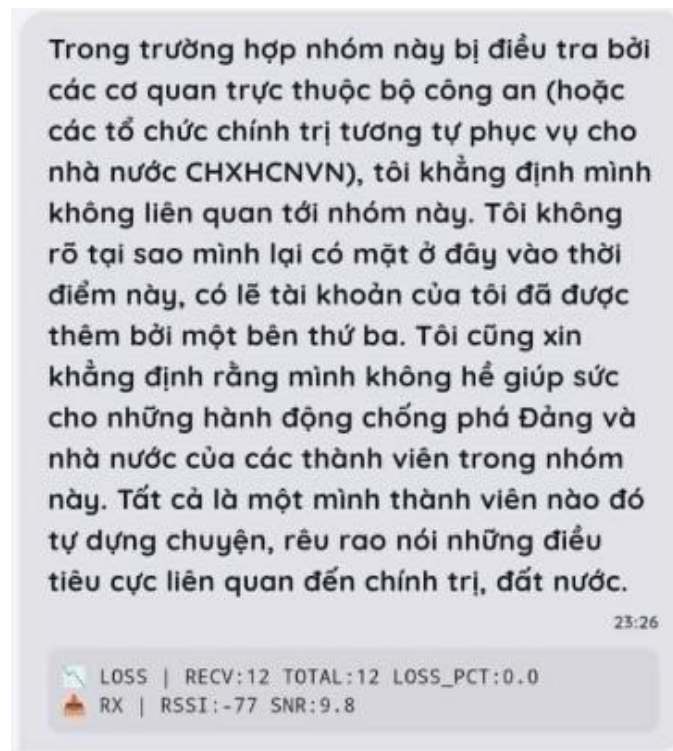
6.2.1 Bài đo xuyên vật cản

Ở bài đo này, hai node sẽ được đặt trong và ngoài phòng. Phòng được đóng cửa kín. Mục tiêu là xác định ảnh hưởng của vật cản đến thông số RSSI/SNR



Hình 6.1. Kết quả đo xuyên 1 bức tường

Tiếp theo, di chuyển 1 node lên tầng 5, node còn lại để ở tầng 2 (trong phòng)



Hình 6.2. Kết quả đo giữa tầng 2 và tầng 5

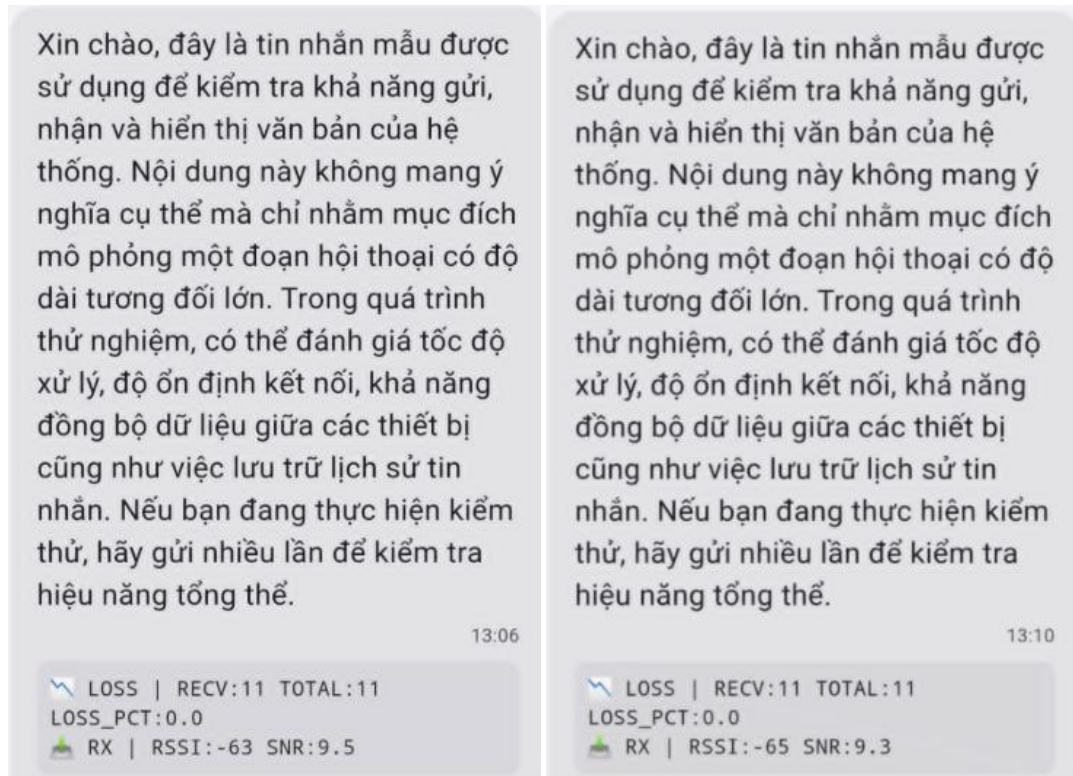
Kịch bản xuyên tường (Hình 6.2): RSSI đo được là -63 dBm và SNR đạt 10.0 dB.

Kịch bản truyền từ tầng 2 lên tầng 5 (Hình 6.3): RSSI suy giảm xuống -77 dBm do tín hiệu phải xuyên qua nhiều lớp sàn bê tông cốt thép. Tuy nhiên, SNR vẫn duy trì ở mức 9.8 dB.

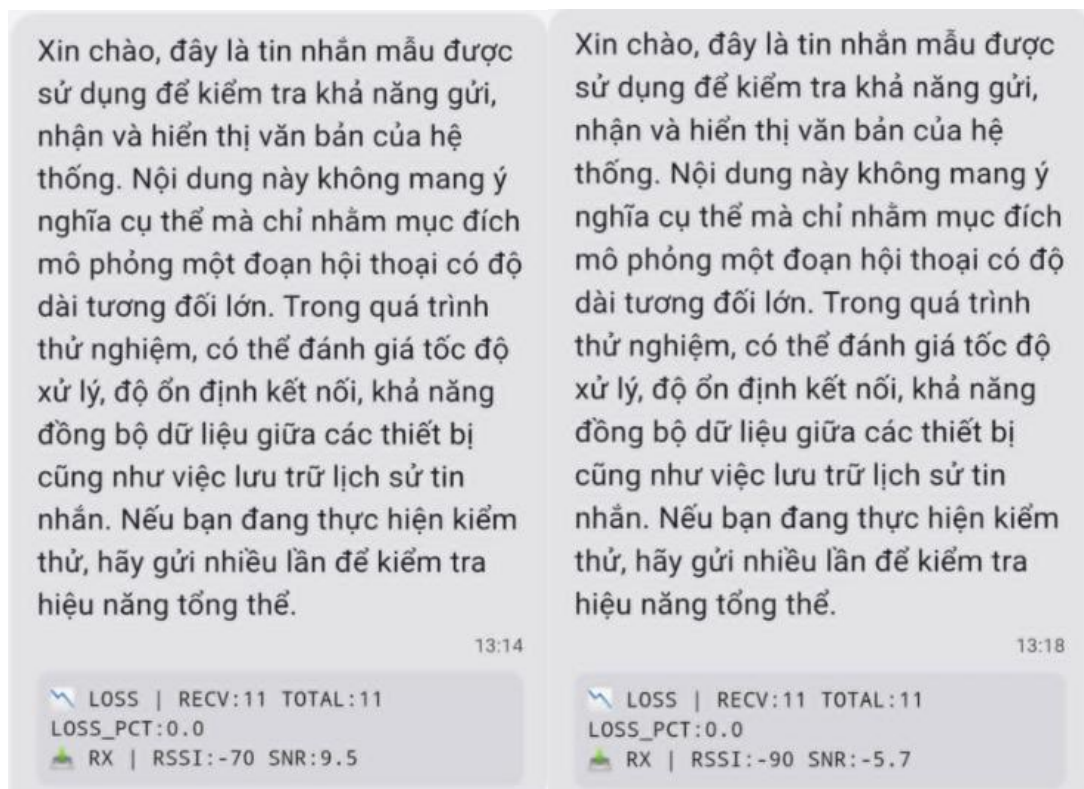
Nhận xét: So với kịch bản xuyên tường, cường độ tín hiệu đã suy giảm khoảng 14 dB khi truyền qua ba tầng lầu, phản ánh ảnh hưởng của suy hao xuyên vật cản. Mặc dù vậy, mức RSSI -77 dBm vẫn cao hơn đáng kể so với độ nhạy thu của chip SX1278 (có thể đạt dưới -120 dBm tùy cấu hình). Đồng thời, giá trị SNR gần 10 dB cho thấy tín hiệu vẫn nổi bật so với nền nhiễu và có chất lượng thu rất tốt. Kết quả này chứng minh rằng liên kết LoRa vẫn hoạt động ổn định trong môi trường trong nhà nhiều tầng, với biên độ dự phòng liên kết (link margin) lớn và xác suất lỗi gói tin thấp.

6.2.2 Bài đo trên khoảng cách lớn tương đối ổn định

Ở bài đo này, 2 node sẽ được đặt ở 2 vị trí với môi trường tương đối ổn định, ít cây và vật cản, có thể nhìn thấy được nhau. Mục tiêu là xác định ảnh hưởng của khoảng cách đến các thông số RSSI/SNR.



Hình 6.3. Kết quả đo với khoảng cách 10m (trái) và 20m (phải)



Hình 6.4. Kết quả đo với khoảng cách 50m (trái) và 100m (phải)

Nhận xét: Kết quả thực nghiệm cho thấy khi khoảng cách truyền tăng từ 10 m lên 100 m, cường độ tín hiệu thu (RSSI) giảm từ khoảng -63 dBm xuống -90 dBm, tương ứng mức suy hao khoảng 27 dB. Đây là hiện tượng phù hợp với quy luật suy hao đường truyền vô tuyến khi khoảng cách giữa bộ phát và bộ thu tăng lên.

Mặc dù RSSI suy giảm đáng kể, hệ thống vẫn nhận thành công toàn bộ 11/11 gói tin ở cả bốn kịch bản thử nghiệm, cho thấy liên kết truyền thông duy trì được độ tin cậy cao. Đặc biệt, tại khoảng cách 100 m, giá trị SNR giảm xuống -5.7 dB. Điều này cho thấy tín hiệu thu đã tiến gần hơn tới mức nhiễu nền của môi trường truyền dẫn. Tuy nhiên, nhờ sử dụng kỹ thuật điều chế Chirp Spread Spectrum (CSS), công nghệ LoRa vẫn có khả năng giải mã chính xác dữ liệu ngay cả khi SNR đạt giá trị âm.

Ngoài ra, giá trị SNR mang giá trị âm tại khoảng cách 100m. Nguyên nhân chủ yếu có thể đến từ suy hao đường truyền, hiệu ứng đa đường (Multipath) trong môi trường thực tế hoặc các nguồn nhiễu vô tuyến cùng băng tần.

Với mức RSSI thấp nhất đo được là -90 dBm, tín hiệu vẫn còn cách khá xa ngưỡng độ nhạy thu của SX1278 (thường từ -120 dBm đến dưới -130 dBm tùy cấu hình). Điều này cho thấy hệ thống vẫn còn một biên độ dự phòng liên kết (Link Margin) đáng kể, đủ để đảm bảo khả năng hoạt động ổn định khi khoảng cách truyền tiếp tục tăng hoặc khi môi trường xuất hiện thêm các yếu tố gây suy hao.

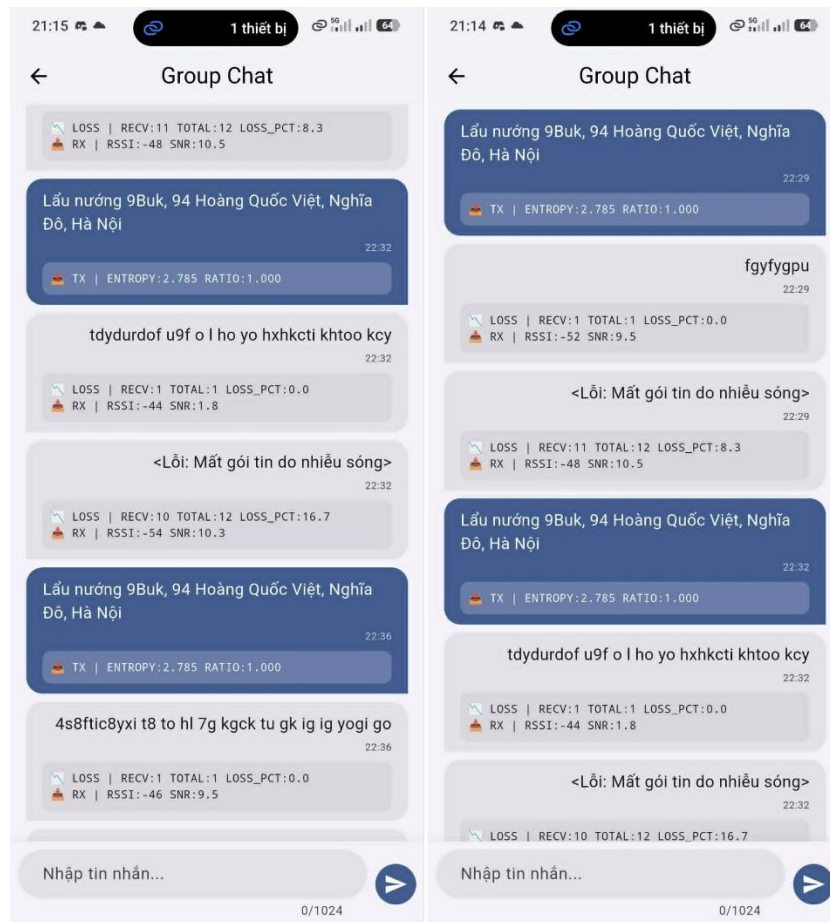
6.3 Đánh giá Thuật toán CSMA/CA và Tỷ lệ mất gói (Packet Loss)

Đây là phép thử quan trọng nhất để chứng minh sự hiệu quả của cơ chế Hàng đợi (RX/TX Queue) và Lùi ngẫu nhiên (Backoff) mà nhóm đã thiết kế ở Chương 4.

Kịch bản Bão Broadcast (Broadcast Storm): Cả 3 máy cùng phát sóng ngay khi nhận lệnh, node 1 và node 2 đóng vai trò gây nhiễu để test, node 3 gửi một đoạn text dài để hệ thống chia thành các packet. Test khoảng cách giữa các node là 5 mét.

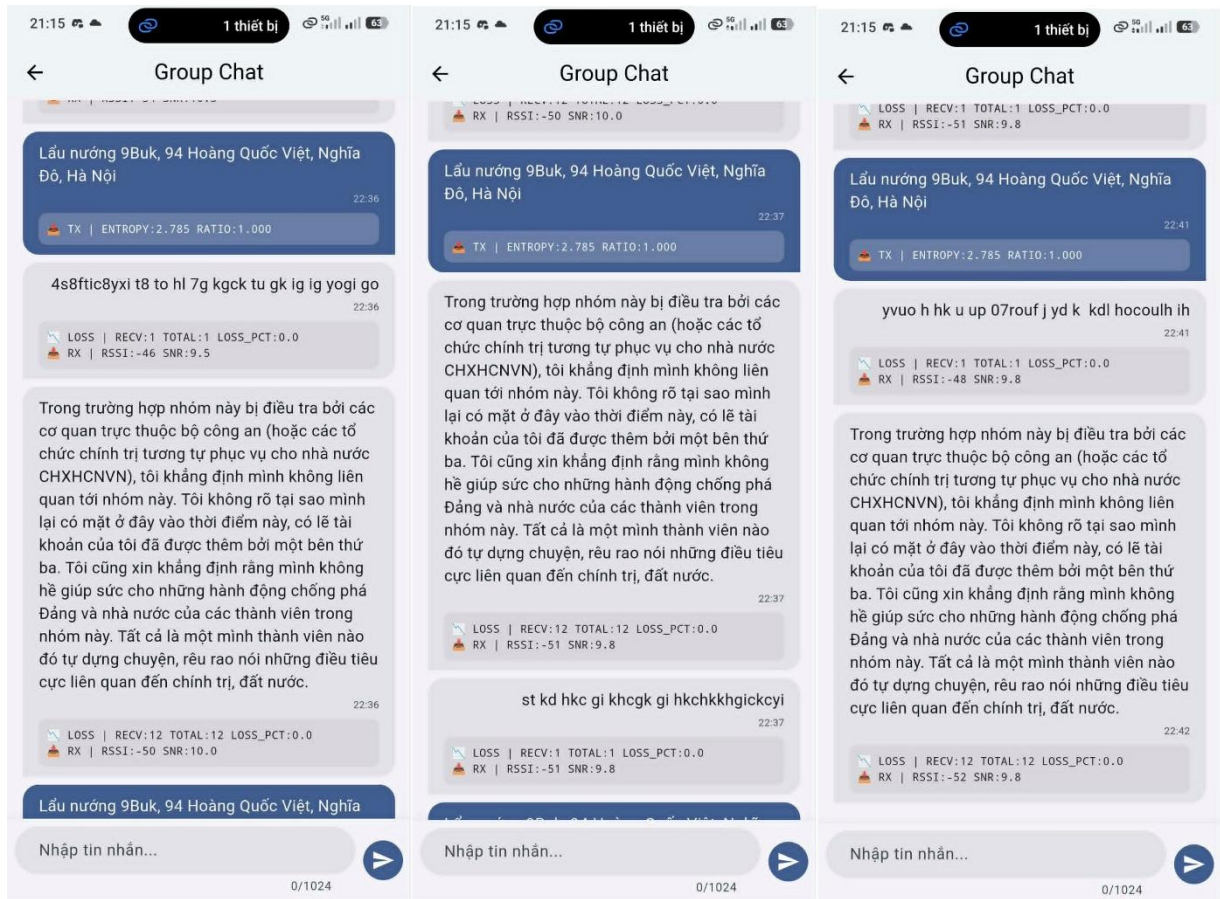
Trường hợp 1: Tắt thuật toán CSMA/CA (Gửi trực tiếp bằng delay):

- Kết quả đo được: Node 1 và 2 gửi thành công, node 3 bị packet loss.
- Tỷ lệ Packet Loss: Như trong ảnh, không thể ghép lại thành tin hoàn chỉnh



Hình 6.5. Test trường hợp tắt thuật toán CSMA/CA

- **Trường hợp 2: Bật thuật toán CSMA/CA Backoff và RX Queue:**
 - Thuật toán xử lý: trước khi gửi thì các gói tin sẽ được đưa vào RX Queue, sau đó có khi có một node phát, các node còn lại sẽ chuyển sang chế độ nghe, sau khi node đó phát xong, các node sẽ random thời gian để chuyển sang trạng thái phát, trong lúc đó, nếu hết thời gian random mà vẫn thấy trên kênh truyền có tín hiệu gửi đến thì ngay lập tức node đó sẽ quay trở lại trạng thái nghe, lặp lại đúng chu trình như ban đầu.
 - Kết quả: Cả 3 máy đều nhận đủ 100% số mảnh của nhau theo thứ tự tuần tự.
 - Tỷ lệ Packet Loss: 0%.



Hình 6.6. Test trường hợp bất CSMA/CA

Kết luận: Cơ chế Hàng đợi bất đồng bộ kết hợp CSMA/CA Random Backoff đã khắc phục triệt để điểm yếu cố hữu của mạng Mesh Flooding.

6.4 Đánh giá Độ trễ khứ hồi (Delay / RTT)

Chỉ số cuối cùng được đo là Round Trip Time (Thời gian từ lúc gửi gói DATA đến lúc nhận được gói ACK báo về).

Bài đo được chia làm 2 phần, phần 1 đo RTT với 2 node gửi trực tiếp cho nhau. Phần 2 đo RTT với 3 node, trong đó 2 node đảm nhận gửi và thu, một node làm trung gian.

RTT sẽ không được đo trong trường hợp hệ thống phát BOARDCAST, do cơ chế phát này không có cơ chế ACK để tránh bão ACK (ACK storm).

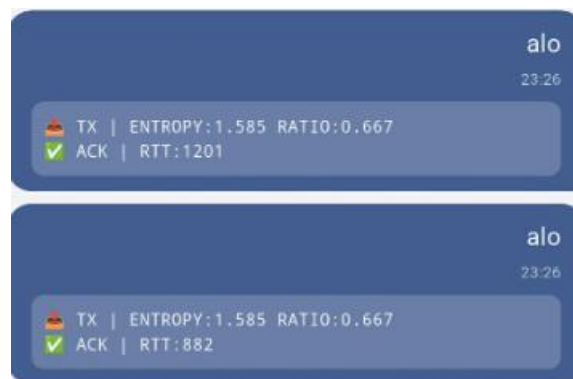
6.4.1 Trường hợp gửi trực tiếp giữa 2 node



Hình 6.7. Kết quả đo RTT ở khoảng cách 50m(trái) và 100m(phải)

Truyền trực tiếp 1 chặng (Node 1 -> Node 2): RTT trung bình = 163 -168 ms.

6.4.2 Trường hợp giao tiếp có node trung gian



Hình 6.8. Kết quả đo RTT ở khoảng cách 200m có relay node

- Truyền qua 2 chặng (Node 1 -> Node 2 -> Node 3): RTT trung bình = 800-1200 ms.

Độ trễ tăng cao qua các chặng do tại mỗi node trung gian, hệ thống phải mất thời gian giải mã, kiểm tra trùng lặp (Deduplication), cộng với thời gian cố ý lùi lại (Backoff) để chống đụng độ trước khi đẩy gói tin đi tiếp. Trong các ứng dụng nhắn tin văn bản không yêu cầu Real-time gay gắt, độ trễ từ 1-2 giây này là hoàn toàn có thể chấp nhận được để đánh đổi lấy vùng phủ sóng rộng gấp 3 lần.

7 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương cuối cùng này tổng kết lại những thành quả mà Đồ án đã đạt được, đồng thời đưa ra những góc nhìn phân biện về các hạn chế còn tồn tại và vạch ra lộ trình nâng cấp hệ thống trong tương lai để đưa sản phẩm tiệm cận với mức ứng dụng thương mại.

7.1 Đánh giá những kết quả đạt được

Thông qua việc phân tích, thiết kế và triển khai thực nghiệm hệ thống nhắn tin đa chặng qua mạng LoRa Mesh, đề tài đã hoàn thành xuất sắc các mục tiêu nghiên cứu ban đầu và chứng minh được sức mạnh của việc áp dụng các lý thuyết thông tin vào môi trường viễn thông thực tế.

Cụ thể, hệ thống đã đạt được các tính năng ưu việt sau:

1. **Thiết lập thành công Mạng Lưới (Mesh Network) 3 Node:** Giải quyết thành công bài toán "Điểm mù sóng" (Dead zones). Việc cấu hình linh hoạt cho phép bất kỳ Node nào cũng có thể đóng vai trò làm Relay chuyển tiếp sóng xuyên tường (NLOS) mà không cần chạm đến hạ tầng cáp quang hay Wi-Fi.
2. **Ứng dụng thực tiễn của Lý thuyết thông tin (Mã hóa Huffman):** Giảm thiểu thành công trung bình từ 25% đến 60% băng thông cho các văn bản chuẩn tiếng Anh thông qua việc nén Entropy cấp độ Bit. Điều này trực tiếp làm giảm thời gian On-Air, tiết kiệm pin cho thiết bị. Hơn thế, việc cấu hình Cơ chế Fallback giúp hệ thống chống chịu tốt với sự khác biệt của bộ mã Unicode (Tiếng Việt).
3. **Hệ thống phòng thủ Đụng độ Toàn diện (CSMA/CA & RX Queue):** Giải quyết triệt để vấn đề sụp đổ mạng lưới do Bão Broadcast (Broadcast Storm). Thực nghiệm cho thấy thuật toán lùi ngẫu nhiên (Random Backoff) kết hợp cảm nhận Carrier Sensing đã đưa tỷ lệ rớt gói (Packet Loss) từ mức thảm họa 75% về con số lý tưởng 0%.
4. **Tích hợp đa nền tảng:** Kết hợp mượt mà giữa phần cứng nhúng (ESP32) thông qua module Bluetooth Asynchronous với phần mềm di động hiện đại (App Flutter), tạo ra một trải nghiệm người dùng (UX) liền mạch như đang sử dụng một ứng dụng nhắn tin thương mại (Zalo/Messenger).

7.2 Các hạn chế còn tồn tại

Bên cạnh các thành quả rực rỡ, dưới góc độ kỹ thuật khắt khe, hệ thống vẫn mang một số khiếm khuyết nội tại cần được nhìn nhận một cách khách quan:

1. **Thuật toán nén tĩnh Huffman chưa tối ưu cho tiếng Việt:** Do giới hạn bảng mã ở chuẩn ASCII (128 ký tự), thuật toán không thể nén các văn bản sử dụng chuẩn UTF-8 (tiếng Việt có dấu). Khi gặp các chuỗi này, hệ thống phải lùi về sử dụng cơ chế truyền thô nguyên bản (LORA_CODEC_NONE), gây lãng phí băng thông có thể nén.

2. **Định tuyến Ngập lụt (Flooding) gây dư thừa lưu lượng:** Dù đã có CSMA/CA phân luồng và bảng seenPackets chống vòng lặp, việc truyền Broadcast vẫn khiến sóng RF bị dội đi dội lại nhiều lần không cần thiết ra các hướng không có Node đích, gây lãng phí tài nguyên kênh truyền tổng (Channel Capacity).
3. **Tắc nghẽn ngầm do Delay:** Để bảo vệ tính toàn vẹn của các gói Broadcast bị phân mảnh, hệ thống phải sử dụng hàm chặn delay(1500). Điều này vô tình làm tăng độ trễ khứ hồi RTT (lên đến xấp xỉ 2 giây/chặng) và làm hệ thống không đáp ứng được các ứng dụng thời gian thực (Real-time Applications) như truyền âm thanh hay đo lường cảm biến tốc độ cao.

7.3 Định hướng phát triển trong tương lai

Từ các điểm hạn chế đã nêu, nhóm vạch ra lộ trình nâng cấp (Roadmap) phiên bản 2.0 cho hệ thống như sau:

1. **Áp dụng Thuật toán Định tuyến Nhảy (AODV - Ad hoc On-Demand Distance Vector):** Thay thế cho cơ chế Flooding "ngây thơ". Các Node sẽ tự động dò đường và ghi nhớ "Tuyến đường ngắn nhất" (Shortest Path) từ người gửi tới người nhận, biến luồng Broadcast tốn kém thành luồng Unicast bảo mật và siêu tốc. Điều này sẽ chấm dứt tình trạng lãng phí băng thông vô tuyến và hạ độ trễ xuống ngưỡng mili-giây.
2. **Triển khai Công nghệ Ngủ sâu (Deep Sleep & Wake-on-Radio):** Thiết kế lại kiến trúc phần cứng, cho phép vi điều khiển ESP32 tắt nguồn gần như hoàn toàn. Module LoRa SX1278 sẽ sử dụng chân ngắt DIO0 đánh thức hệ thống dậy chỉ khi phát hiện có dải xung Preamble bay ngang qua. Bản nâng cấp này có thể giúp hệ thống chạy liên tục hàng năm trời chỉ với vài viên pin tiểu AA, mở ra cánh cửa tiến vào hệ sinh thái Internet Vạn Vật (IoT - Internet of Things) dân dụng đích thực.